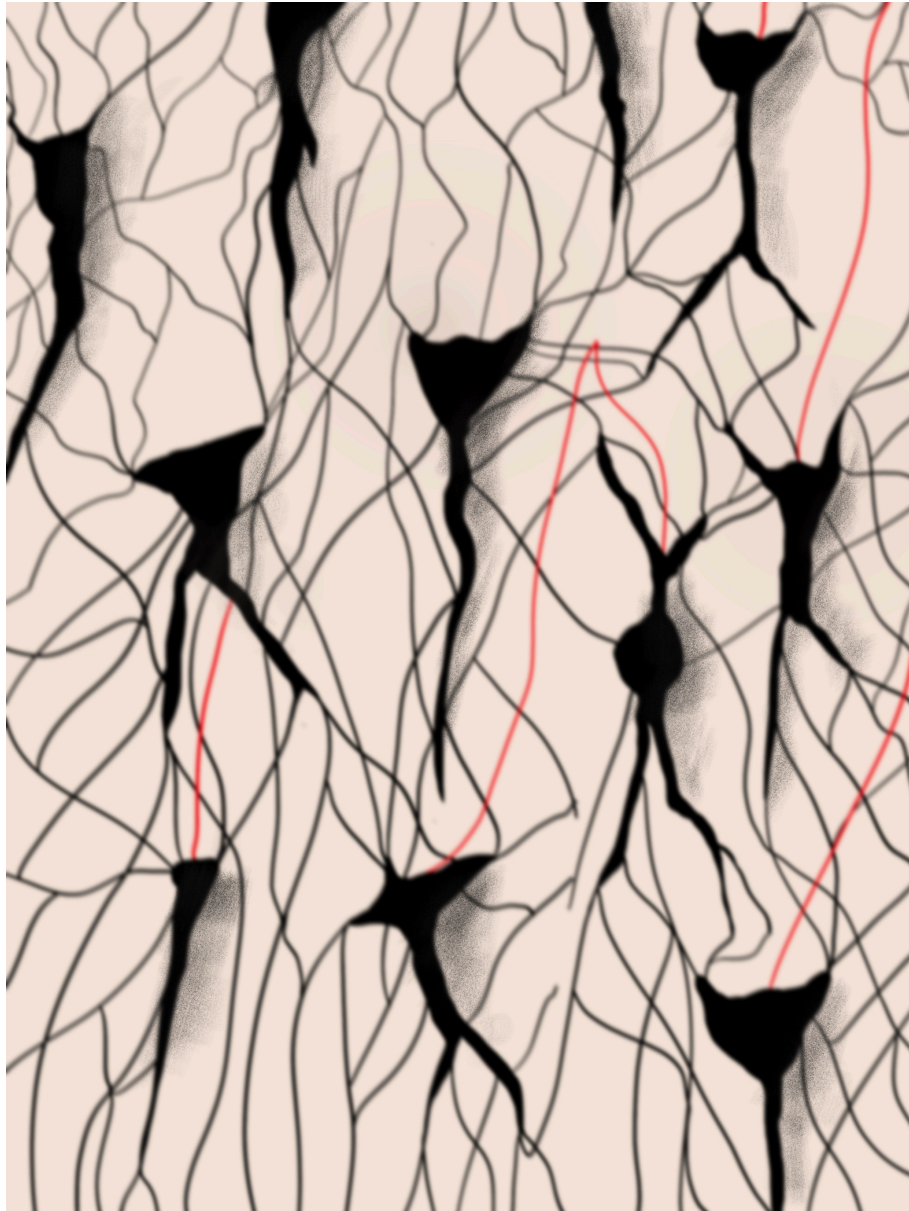


A Technical Notebook on Computational Neuroscience

Natalia T. Cónsul

Based on lectures by **Prof. Claudia Clopath**

Imperial College London



Contents

1	Simplest model: The passive membrane	5
1.1	Linear differential equation governing the passive membrane	5
1.2	Review: Methods for solving differential equations	6
1.3	Formal solution: differential equation derivation	7
2	From the passive membrane to spikes: Leaky integrate-and-fire model	9
2.1	The components of the LIF model	9
2.2	Fixed-point analysis of the LIF neuron	9
3	From circuits to biophysics: The origin of membrane voltages	12
3.1	Deriving Nernst equation	12
4	Biophysical modeling	13
4.1	The Hodgkin–Huxley model: A biophysical basis for action potentials	13
4.2	How ion channels generate currents	13
4.3	Channel dynamics: Activation and inactivation gates	14
5	Introduction to phase-plane analysis	16
5.1	Reducing the Hodgkin–Huxley model to a phase plane	16
5.2	Stability: Graphical intuition	16
5.3	Stability: Mathematical view	17
6	Simplified models of neuronal excitability	18
6.1	The FitzHugh–Nagumo model	18
6.2	The Morris–Lecar model	23
7	Generalized integrate-and-fire models	26
7.1	From LIF to nonlinear models	26
7.2	Adaptive integrate-and-fire models	26
8	The Cable Equation	28
8.1	Deriving the cable equation	28
9	Model of a Synapse	30
9.1	Modeling synapses: Conductances and postsynaptic currents	30
9.2	Differences between excitatory and inhibitory synapses	30
10	Variability and Noise in Neural Activity	32
10.1	Noise in the inputs	32
10.2	Stochastic spike arrival	32
10.3	Noise in the output	32
11	Characterizing Neural Activity	33
12	A neural population perspective: from single neurons to rate models	34
12.1	The mean firing rate	34
12.2	The gain function	34
12.3	The rate equation	35
13	Recurrent rate networks	36
13.1	Bistability	36
14	Interacting rate networks	38
14.1	Excitatory–excitatory interactions: cooperative bistability	38
14.2	Inhibitory–inhibitory interactions: competitive dynamics	39
14.3	Excitatory–inhibitory interactions: balanced networks	39

15 Decision making	42
15.1 The random-dot motion task	42
15.2 A three-population competitive E–I network	42
15.3 Equations of the decision circuit and model reduction	43
15.4 Dynamics of decision making	44
16 Continuous Population Models	47
16.1 The ring network	47
16.2 Rate equation on the ring	47
16.3 Cosine-shaped connectivity	47
16.4 The original ring model	48
17 Neural networks of spiking neurons	50
17.1 Single-neuron dynamics	50
17.2 Large recurrent networks	51
17.3 Synaptic input: mean and fluctuations	51
17.4 Weak vs. strong coupling	52
17.5 The balanced state	52
18 Unsupervised learning	53
18.1 Introduction to learning and memory	53
18.2 Hebbian learning: “cells that fire together, wire together”	53
18.3 Rate-based formalism	53
18.4 Hebbian learning extracts input correlations	54
18.5 Why pure Hebbian learning is unstable	55
18.6 The covariance rule: Hebbian learning with baseline subtraction	56
18.7 Oja’s rule: Hebbian learning with normalization	56
18.8 The BCM rule: Hebbian learning with a sliding threshold	57
18.9 Spike-Timing-Dependent Plasticity	59
19 Supervised Learning	62
19.1 The Perceptron	62
19.2 Geometric interpretation	63
19.3 Relation to biological learning	63
20 Reinforcement Learning	64
20.1 Classical conditioning	64
20.2 The Rescorla–Wagner model	64
20.3 Blocking and limitations	65
20.4 Temporal-Difference learning	65
21 Hopfield networks and associative memory	66
21.1 Binary neurons and interactions	66
21.2 Neuron update rule	66
21.3 Detour: connection to magnetism	67
21.4 Pattern storage in Hopfield networks	67
21.5 Pattern retrieval	69

Introduction

Neurons in our body are a bit like tiny biological circuits. Each neuron receives signals from other neurons, processes those signals, and then sends a new signal onward. The brain and spinal cord are built from billions of these units connected together, forming large networks. Everything we do, seeing, moving, learning, and thinking, emerges from the activity of these neural circuits.

Imagine you want to grab the cup of coffee next to you while reading this notebook. You look at it, decide to reach for it, and your arm and fingers smoothly move into place. You do not consciously calculate forces, trajectories, or muscle activations, yet the movement is precise and flexible. This seemingly simple action already requires many neurons to work together. Neurons in your eyes respond to the light reflected from the cup, neurons in your brain help plan and initiate the reach, and neurons in your spinal cord translate those commands into detailed patterns of muscle activity. If the cup slips or your hand is slightly off target, the system rapidly corrects itself. All of this relies on neurons communicating with each other.

Neurons communicate using electricity. Inside each neuron there is a small voltage, called the membrane potential, that changes over time. When a neuron receives input from other neurons, this voltage goes up or down. If it becomes large enough, the neuron produces a very fast electrical pulse called a spike. Spikes are brief and look very similar each time, so what matters most is not their exact shape, but when a spike happens and how often spikes occur.

Because neurons communicate using spikes, information in the nervous system is carried by patterns of spikes across time and across many neurons. A single neuron can only do so much, but when many neurons act together, their combined activity can represent sensory information, decisions, and movements. To understand the brain, it is therefore not enough to study neurons in isolation. We must also understand how populations of neurons interact and evolve over time.

Experiments allow us to record neural activity, sometimes from hundreds or thousands of neurons at once. However, recordings alone do not tell us why neurons behave the way they do or how their activity gives rise to behavior. This is where models become useful. A model is a simplified description of a neuron or a neural circuit. Much like a model of an electronic circuit, it helps us understand how inputs are transformed into outputs.

In computational neuroscience, we use mathematical models to describe how neurons change over time and how spikes are generated. Some models aim to be biologically detailed, while others are intentionally simple. Simple models are easier to analyze and often capture the core principles we care about, while more detailed models help explain how specific biological mechanisms shape neural behavior.

This notebook uses models to build intuition about how neurons and neural circuits work. The goal is not to memorize equations, but to develop a way of thinking about the nervous system as a dynamical system composed of interacting circuits. By the end, you should feel comfortable using models to explore how neural activity arises, how it changes over time, and how it ultimately supports behavior, including something as ordinary, and as remarkable, as reaching for a cup of coffee.

1 Simplest model: The passive membrane

The passive membrane describes the behavior of a neuron's membrane when no active ion channels are engaged. This means that the behavior of the membrane can be represented as a **linear system**. The main idea behind the passive membrane model is that the membrane potential evolves over time in response to external currents. This model can be interpreted as an electrical circuit with several components:

- Resting Potential, u_{rest} : This is the starting voltage across the membrane, when the neuron is at rest.
- Resistor, R : The membrane has resistance, which slows the movement of ions across it.
- Capacitor, C : The membrane stores charge as a battery. This stored charge creates the membrane potential (voltage).
- Input current, $I(t)$: When the neuron receives a signal, current flows in, adding charge to the capacitor, and changing the membrane potential.

Importantly, the circuit in the passive membrane model represents the **subthreshold regime** of a neuron (Figure 1). In this regime, the membrane potential changes in response to incoming signals, but **does not reach the threshold required to trigger an action potential**.

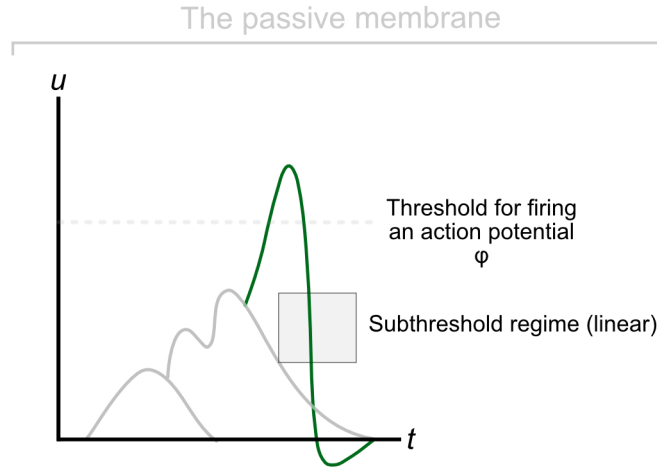


Figure 1: The subthreshold regime of the passive membrane. Voltage responses to synaptic input that do not reach the firing threshold φ remain subthreshold and do not trigger an action potential. In this regime, membrane dynamics are well described by a linear passive model.

1.1 Linear differential equation governing the passive membrane

The **current through the capacitor** is given by the following relationship, where C is the capacitance, Q the charge, and u is the voltage across the capacitor:

$$C = \frac{Q}{u}$$

Taking the derivative with respect to time:

$$C \cdot \frac{du}{dt} = \frac{dQ}{dt}$$

The rate of change of charge with respect to time is the current through the capacitor, I_C :

$$I_C = C \cdot \frac{du}{dt}$$

To calculate the **current that goes through the resistor**, we apply Ohm's Law, where u_R is the voltage drop across the resistor ($u - u_{\text{rest}}$):

$$u_R = R \cdot I_R$$

Now, **combining** both the **capacitor** and **resistor** currents in the circuit, **we obtain the total current**:

$$I(t) = \frac{1}{R} \cdot (u - u_{\text{rest}}) + C \cdot \frac{du}{dt}$$

Multiplying both sides by R to simplify:

$$R \cdot C \cdot \frac{du}{dt} = R \cdot \left(-\frac{1}{R} \cdot (u - u_{\text{rest}}) \right) + R \cdot I(t)$$

Recognizing that $R \cdot C$ is the **time constant** τ (**how quickly the system responds to a change in the input current**), the equation simplifies to:

$$\boxed{\tau \cdot \frac{du}{dt} = -(u - u_{\text{rest}}) + R \cdot I(t)} \quad (1)$$

At this point, we can note that u_{rest} is constant, so $\frac{du_{\text{rest}}}{dt} = 0$.

Therefore, we can ignore u_{rest} in the differential equation, as its derivative does not contribute to the change in membrane potential.

This is the **linear differential equation governing the passive membrane model**, where the membrane potential evolves over time based on the input current and the properties of the membrane (resistance and capacitance).

1.2 Review: Methods for solving differential equations

Step 1: Find the fixed point

The first question we ask is: **Where is the system trying to go?** That target value is called the **fixed point** (or **steady state**). It is defined as the value of u for which the system stops changing:

$$\frac{du}{dt} = 0$$

For our equation:

$$\tau \frac{du}{dt} = -(u - u_{\text{rest}}) + RI(t)$$

setting $\frac{du}{dt} = 0$ gives:

$$u_{\infty} = u_{\text{rest}} + RI(t)$$

So, the **fixed point tells us the voltage the system will eventually reach**. Without identifying it, we would not know the destination of the trajectory.

Step 2: Understand exponential relaxation

Once we have the fixed point u_{∞} , the next step is to describe how the system approaches it. The key idea is that the **rate of change is proportional to the distance from the fixed point**:

- If u is far away, it moves quickly.
- If u is close, it moves slowly.

This proportional shrinking of the difference always produces an exponential approach. Mathematically:

$$\frac{du}{dt} = -\frac{1}{\tau}(u - u_{\infty})$$

Let the distance to the fixed point be:

$$\Delta u(t) = u(t) - u_{\infty}$$

Then the differential equation becomes:

$$\frac{d}{dt} \Delta u(t) = -\frac{1}{\tau} \Delta u(t)$$

This is a **standard exponential decay**, with solution:

$$\Delta u(t) = \Delta u(0)e^{-t/\tau}$$

Here $\Delta u(0)$ is the initial distance from the fixed point:

$$\Delta u(0) = u(0) - u_\infty$$

Since $\Delta u(t) = u(t) - u_\infty$, we write:

$$u(t) - u_\infty = (u(0) - u_\infty)e^{-t/\tau}$$

Finally, solving for $u(t)$ gives:

$$u(t) = u_\infty + (u(0) - u_\infty)e^{-t/\tau}$$

We can rewrite this in a more intuitive form:

$$\boxed{u(t) = u(0)e^{-t/\tau} + (1 - e^{-t/\tau})u_\infty} \quad (2)$$

This expression makes the dynamics clear:

- $e^{-t/\tau}$ is a **forgetting term**. As t increases, $e^{-t/\tau} \rightarrow 0$, so the system gradually forgets its initial value $u(0)$.
- $(1 - e^{-t/\tau})u_\infty$ is a **convergence term**. As t increases, $(1 - e^{-t/\tau}) \rightarrow 1$, so $u(t)$ approaches the fixed point u_∞ .

This means that the system exponentially forgets where it started and exponentially approaches its fixed point.

This equation plays a central role in computational neuroscience. Membrane dynamics, synaptic currents, eligibility traces, and learning rules all rely on the same exponential **forget-and-converge mechanism**.

1.3 Formal solution: differential equation derivation

In practice, you do not need to carry out this full derivation every time. As soon as you see an equation of the form:

$$\frac{du}{dt} \propto -(u - u_\infty)$$

you can immediately recognize that the solution is an **exponential relaxation toward u_∞ with time constant τ** .

The derivation below is included mainly to show where this exponential form comes from.

We begin with the differential equation:

$$\frac{du}{dt} = -\frac{1}{\tau}(u - u_\infty)$$

To solve, we first separate variables:

$$\frac{du}{u - u_\infty} = -\frac{1}{\tau} dt$$

Integrating both sides gives:

$$\int \frac{du}{u - u_\infty} = \int -\frac{1}{\tau} dt$$

so that:

$$\ln |u - u_\infty| = -\frac{t}{\tau} + C$$

Exponentiating both sides removes the logarithm:

$$u - u_\infty = Ae^{-t/\tau}$$

where $A = e^C$ is a constant determined by the initial condition.

At $t = 0$, we know $u(0) = u_0$, so:

$$u_0 - u_\infty = A$$

Putting everything together, the final solution is:

$$u(t) = u_\infty + (u_0 - u_\infty)e^{-t/\tau}$$

which shows explicitly that the membrane potential relaxes exponentially toward the fixed point u_∞ .

Example case: Step input

If the input current suddenly becomes constant ($I(t) = I_1$ for $t \geq 0$), then the fixed point of the system is:

$$u_\infty = u_{\text{rest}} + RI_1$$

From the general solution:

$$u(t) = u_\infty + (u(0) - u_\infty)e^{-t/\tau}$$

we now set the initial condition $u(0) = u_{\text{rest}}$ (the neuron starts at rest). Substituting this into the formula gives:

$$u(t) = u_\infty + (u_{\text{rest}} - u_\infty)e^{-t/\tau}$$

Finally, replacing u_∞ with $u_{\text{rest}} + RI_1$ and simplifying, we obtain:

$$u(t) = u_{\text{rest}} + RI_1(1 - e^{-t/\tau})$$

This expression shows that the membrane potential starts at u_{rest} and rises exponentially toward the new fixed point $u_\infty = u_{\text{rest}} + RI_1$ with time constant τ .

2 From the passive membrane to spikes: Leaky integrate-and-fire model

The Leaky integrate-and-fire (LIF) model builds directly on the passive membrane equation but adds a simple event rule that turns **continuous voltage trajectories into discrete spikes**. A spike refers to a brief, rapid increase and subsequent decrease in the membrane potential, representing the fundamental electrical signal by which neurons communicate.

2.1 The components of the LIF model

(1) Subthreshold dynamics (like the passive membrane)

$$\tau \frac{du}{dt} = -(u - u_{\text{rest}}) + RI(t)$$

Between spikes, the membrane potential integrates the input current and leaks back toward the resting potential u_{rest} with time constant τ .

(2) Threshold condition

$$t^{(f)} : \quad u(t^{(f)}) = \varphi, \quad \dot{u}(t^{(f)}) > 0$$

Whenever the **voltage crosses the threshold** φ from below, the model declares that a spike has occurred at time $t^{(f)}$.

(3) Reset (and refractory period)

$$\lim_{\varepsilon \rightarrow 0^+} u(t^{(f)} + \varepsilon) = u_{\text{rest}} < \theta$$

Immediately after a spike, the potential is reset to u_{rest} . Optionally, it can be held there for an absolute refractory period τ_{ref} , during which the **neuron is temporarily unable to fire another spike**, ensuring a minimum time between consecutive action potentials (just as occurs in real neurons) before integration resumes.

2.2 Fixed-point analysis of the LIF neuron

The key idea is that the firing rate is controlled by how far the fixed point u_{∞} is from threshold, and whether this fixed point is stable or unstable.

- When $I = 0$, the fixed point is the resting potential u_{rest} , which is below threshold φ . This is a **stable fixed point: if the voltage is perturbed slightly, it always relaxes back to u_{rest}** . The neuron therefore remains silent. (See left panel of the Figure 2: the trajectory stays flat at rest).
- When a constant input $I > 0$ is applied, the fixed point shifts upward to:

$$u_{\infty} = u_{\text{rest}} + RI$$

If this new fixed point still lies below threshold, it remains stable and no spikes occur. But if u_{∞} lies above threshold, it becomes **unstable**: the voltage cannot remain there. Instead, **$u(t)$ is driven toward φ , a spike is generated, and the system is reset** (See right panel of the Figure 2). This cycle then repeats.

The larger the input current I , the higher the fixed point u_{∞} . Because $u(t)$ then reaches threshold more quickly, the time between spikes is shorter. As a result, **stronger currents lead to a higher firing frequency**. Conversely, if the input is too weak, the fixed point u_{∞} remains below threshold, the resting point is stable, and the neuron stays silent.

In summary:

More current $I \Rightarrow u_{\infty}$ closer to or above threshold $\Rightarrow$ faster threshold crossings $\Rightarrow$ higher firing rate.

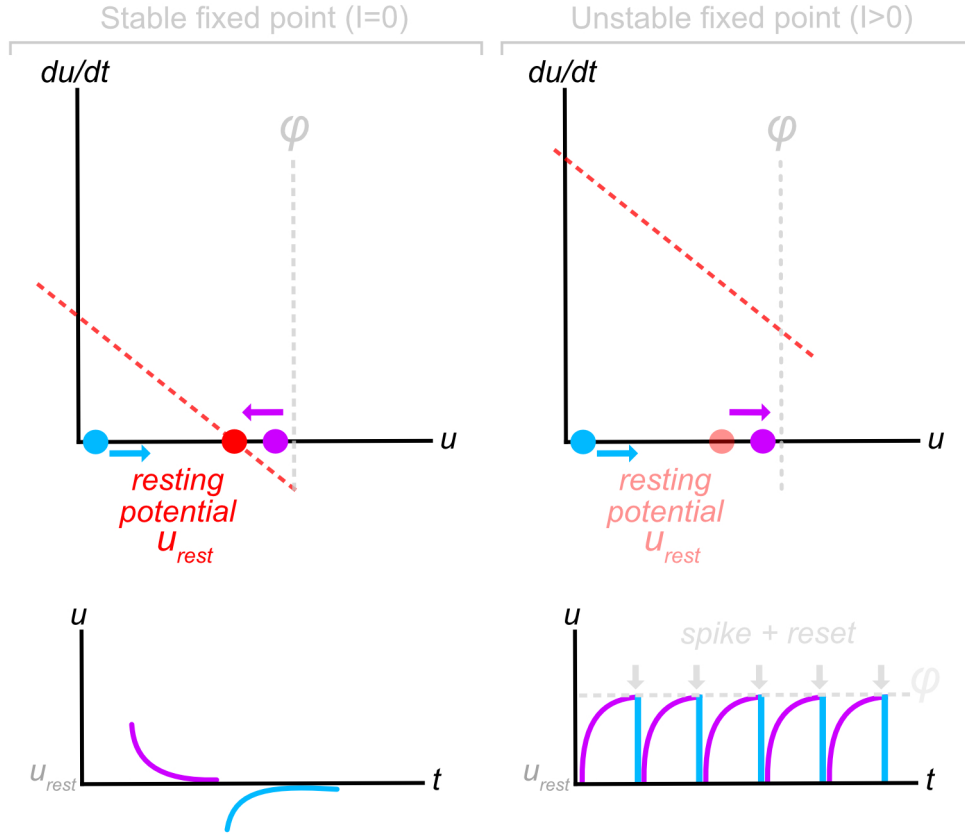


Figure 2: Stable ($I = 0$) vs. unstable ($I > 0$) fixed points in the LIF model. Without input, u_{rest} is stable and the voltage stays at rest. With constant input, the fixed point moves above threshold, becomes unstable, and produces repetitive spiking.

General viewpoint The LIF model can be seen as a special case of a more general system (this will be discussed in more detail in Chapter 7).

$$\tau \frac{du}{dt} = F(u, t)$$

where $F(u, t)$ is an arbitrary function of the membrane potential and time. What matters for spiking behavior is not the exact form of $F(u, t)$, but rather the presence of two key ingredients (Figure 3):

1. A **stable fixed point** u_{rest} that attracts the dynamics in the absence of input ($I = 0$).
2. A **threshold–reset rule** that generates a spike once $u(t)$ reaches θ and then resets the potential to u_{rest} .

Here it is important to distinguish biology from modeling: in real biophysical neuron models (e.g. Hodgkin–Huxley or FitzHugh–Nagumo, models that are described in the next sections), threshold corresponds to an unstable fixed point. Below it, trajectories decay back to the stable resting state; above it, they run away, triggering a spike. In contrast, the LIF model deliberately **keeps the rest potential stable** and **replaces the unstable fixed point with a simplified threshold–reset mechanism**.

As long as these two ingredients are present, any sufficiently strong input that drives the system across threshold will produce a regular train of spikes. The LIF model is therefore best understood as the simplest representative of a broader family of threshold–reset neuron models.

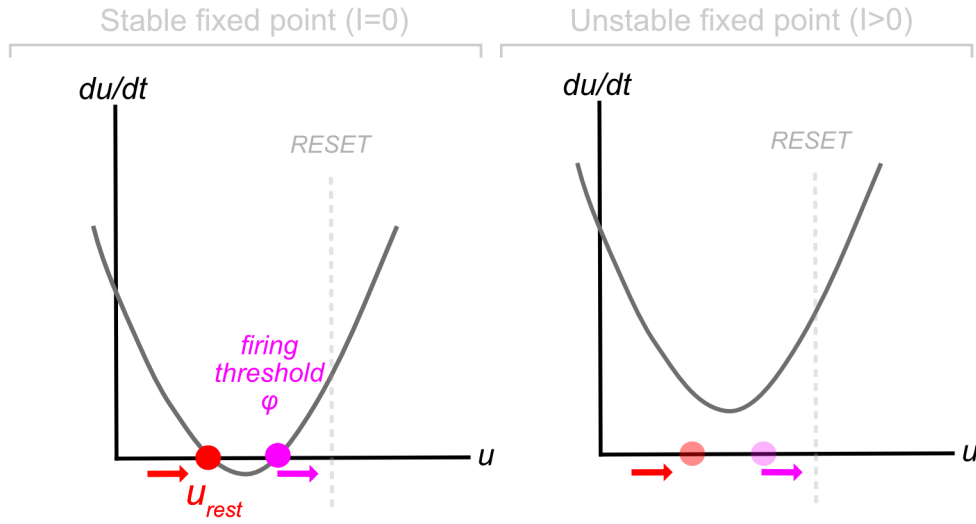


Figure 3: General view of fixed-point behavior. For $I = 0$, u_{rest} represents a stable fixed point, where the membrane potential returns to rest after small perturbations. When constant input is applied ($I > 0$), the fixed point becomes unstable, driving the voltage toward the firing threshold φ , where it is reset, producing repetitive spiking.

3 From circuits to biophysics: The origin of membrane voltages

Up to now, we have modeled the neuron's membrane as a simple electrical circuit, with resistors, capacitors, and input currents that determine how the voltage evolves. But **what is the origin of this voltage in real biology?** To answer this, we must go from circuits to biophysics.

The neuronal membrane separates two solutions with different ionic concentrations. Ion pumps actively move ions across the membrane, while ion channels allow them to flow back. As a result, **there is more sodium (Na^+) outside the cell and more potassium (K^+) inside. This imbalance is crucial: without it, there would be no stored energy and therefore no voltage.** The gradients provide a constant driving force for ions to move, which is what makes signaling possible.

3.1 Deriving Nernst equation

To describe how these forces translate into voltages, we use the **Boltzmann distribution**, which **links energy to the probability of finding an ion at a particular location**. The energy of a charged particle at a location x with electric potential $u(x)$ is:

$$E(x) = q u(x)$$

where q is the charge.

According to Boltzmann, the probability of a state of energy E is:

$$p(E) \propto \exp\left(-\frac{E}{kT}\right)$$

where k is the Boltzmann constant and T is the absolute temperature.

This means the relative density of ions at two positions x_1 and x_2 is:

$$\frac{n(x_1)}{n(x_2)} = \exp\left[-\frac{q(u(x_1) - u(x_2))}{kT}\right]$$

Rearranging gives the potential difference:

$$\Delta u = u(x_1) - u(x_2) = \frac{kT}{q} \ln \frac{n(x_2)}{n(x_1)} \quad (3)$$

This expression already shows how concentration ratios produce a voltage across the membrane.

To make it specific to ions, we note that $q = ze$ where z is the ion valence and e the elementary charge.

$$E_{\text{rev},i} = \frac{kT}{ze} \ln \frac{[i]_{\text{out}}}{[i]_{\text{in}}} \quad (4)$$

This voltage is called the **reversal potential**, because it is the point at which the **net current for ion i reverses direction**.

The derivation works for any ion species. In practice, we often use potassium (K^+) and sodium (Na^+) as examples, since they dominate the resting potential of neurons. But the same principle applies to chloride (Cl^-), calcium (Ca^{2+}), and others.

4 Biophysical modeling

In 1952, **Alan Hodgkin and Andrew Huxley** published a series of experiments on the giant squid axon, laying the foundation of modern electrophysiology [1, 2]. They described how input currents change the membrane potential through a combination of capacitive charging and ionic currents, and proposed a **mathematical model that explained the initiation and shape of action potentials**. For this work, they were awarded the Nobel Prize in Physiology or Medicine in 1963.

4.1 The Hodgkin–Huxley model: A biophysical basis for action potentials

When an input current $I(t)$ is injected into the neuron, part of it charges the membrane capacitance C , while the rest leaks through ion channels. This can be represented by an electrical circuit consisting of:

- A capacitor C , representing the membrane’s ability to store electrical charge.
- Variable conductances, corresponding to ion channels whose resistance changes as they open or close.
- Batteries, representing ionic concentration gradients and characterized by their reversal potentials E_i .

The total current balance is:

$$I(t) = I_C + I_K + I_{Na} + I_L$$

where the capacitive current is:

$$I_C = C \frac{du}{dt}$$

The last three terms describe the ionic currents through potassium (K), sodium (Na), and leak (L) channels. Modern variants of the Hodgkin–Huxley model often add more ion types or detailed channel kinetics, but here we keep the classical version for clarity.

4.2 How ion channels generate currents

Hodgkin and Huxley described each ionic current using **Ohm’s law**:

$$I_i = g_i(u - E_i)$$

where g_i is the conductance (inverse of resistance) of channel i , u is the membrane potential, and E_i is the reversal potential for ion i . For the leak current:

$$I_L = g_L(u - E_L)$$

The leak current represents the small, steady flow of ions that always occurs through non-gated (always open) channels. Unlike voltage-gated sodium or potassium channels, these leak channels are not controlled by the membrane potential. They allow a constant exchange of ions that helps the neuron maintain its resting potential and return to it after small perturbations. Here g_L is constant, while sodium and potassium conductances (g_{Na} , g_K) vary with time and voltage.

Substituting into the current balance gives:

$$\boxed{C \frac{du}{dt} = -g_K(u - E_K) - g_{Na}(u - E_{Na}) - g_L(u - E_L) + I_{\text{ext}}} \quad (5)$$

The parameters E_{Na} , E_K , and E_L are the reversal potentials are shown in Table 1. These values can be used to understand the upcoming section about channel dynamics.

Table 1: Reversal potentials (E_x) and conductances (g_x) for Na, K, and leak (L) channels.

x	E_x [mV]	g_x [mS/cm ²]
Na	55	40
K	-77	35
L	-65	0.3

4.3 Channel dynamics: Activation and inactivation gates

In the equivalent circuit, conductances g_{Na} and g_K were introduced to describe how **sodium and potassium channels** carry current. However, these conductances are not fixed resistors: they **change with both voltage and time, because channels can open or close**.

To capture this, Hodgkin and Huxley introduced **gating variables**. Each gating variable represents the **probability that a particular subunit of an ion channel is open**. Ion channels are not single doors; they are made of several **protein subunits** that must be in the “open” configuration for ions to pass. Different ions use channels with different structures, so their subunits open and close with different kinetics. Hodgkin and Huxley captured these differences by assigning separate gating variables to each type of subunit:

$$g_K = \bar{g}_K n^4, \quad g_{Na} = \bar{g}_{Na} m^3 h$$

Here:

- n = activation gate for K⁺ channels (four independent subunits; all must be open for conduction).
- m = activation gate for Na⁺ channels (three fast subunits).
- h = inactivation gate for Na⁺ channels (one slower subunit).

Thus n^4 is the probability that all four potassium subunits are open, and $m^3 h$ is the probability that the sodium channel is open and not inactivated. Substituting these expressions into the current balance equation gives the full Hodgkin–Huxley model:

$$C \frac{du}{dt} = -\bar{g}_K n^4 (u - E_K) - \bar{g}_{Na} m^3 h (u - E_{Na}) - g_L (u - E_L) + I_{\text{ext}} \quad (6)$$

This equation combines the circuit analogy with gating dynamics, making it possible to explain the shape of the action potential.

Dynamics of gating variables

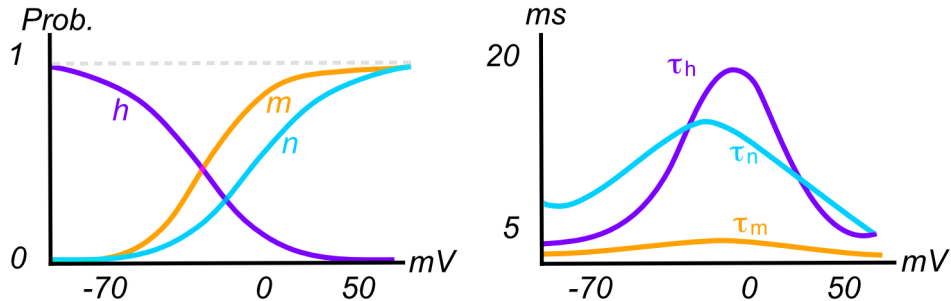


Figure 4: Voltage dependence of Hodgkin–Huxley gating variables. **Left:** Steady-state activation and inactivation curves $x_\infty(u)$. Sodium activation (m) and potassium activation (n) increase with depolarization, while sodium inactivation (h) decreases. **Right:** Corresponding time constants $\tau_x(u)$. Sodium activation is fast (τ_m small), whereas sodium inactivation (τ_h) and potassium activation (τ_n) are slower.

Each gating variable $x \in \{m, h, n\}$ follows a first-order differential equation:

$$\frac{dx}{dt} = \alpha_x(u)(1 - x) - \beta_x(u)x$$

which can be rewritten in the standard forget-and-converge form (Eq. 2):

$$\frac{dx}{dt} = \frac{x_\infty(u) - x}{\tau_x(u)}, \quad x_\infty(u) = \frac{\alpha_x(u)}{\alpha_x(u) + \beta_x(u)}, \quad \tau_x(u) = \frac{1}{\alpha_x(u) + \beta_x(u)}$$

Here, $x_\infty(u)$ is the voltage-dependent steady-state value (the “target” open probability), and $\tau_x(u)$ sets how rapidly the gating variable relaxes toward this target.

Together, the curves in Figure 4 and the equations above describe how each gating variable evolves in time as the membrane voltage changes. We now use this framework to follow the sequence of events during an action potential:

Resting state. At rest ($u \approx -70$ mV), sodium activation is minimal ($m \approx 0$), potassium channels are closed ($n \approx 0$), and sodium inactivation is largely removed ($h \approx 1$). In this state, inward sodium current is suppressed and the membrane remains stable.

Spike initiation (upstroke). As the membrane depolarizes, m and n increase (activation), while h decreases (inactivation). Because τ_m is small, sodium activation is fast, allowing m to rise rapidly and open Na^+ channels. This rapid inward sodium current drives the sharp upstroke of the action potential. In contrast, τ_h and τ_n are larger, so sodium inactivation and potassium activation occur more slowly. This **separation of time scales makes the firing threshold dynamic**: a sufficiently strong or rapid depolarization can trigger a spike before inactivation or repolarization takes over.

Spike termination and refractoriness. The action potential terminates as h closes sodium channels and n opens potassium channels, driving membrane repolarization. The delayed recovery of h and n produces a refractory period, during which the neuron is less excitable and immediate re-firing is suppressed.

Summary. Action-potential generation in the Hodgkin–Huxley model relies on three coordinated processes:

- m : fast sodium activation driving the spike upstroke.
- h : sodium inactivation terminating inward current.
- n : delayed potassium activation repolarizing the membrane.

Their voltage dependence and distinct time scales explain the sharp rise, delayed fall, dynamic threshold, and refractory period of the action potential.

5 Introduction to phase-plane analysis

5.1 Reducing the Hodgkin–Huxley model to a phase plane

The full Hodgkin–Huxley model uses four variables: the membrane voltage u and the gating variables m , h , and n . This forms a **four-dimensional dynamical system**, which cannot be visualized directly. To make it more tractable, we apply two standard reductions to obtain a **two-dimensional** system:

- **Fast sodium activation (m).** The gating variable m evolves extremely quickly (see the time constant in Figure 4). It reaches its steady state almost instantaneously, so we set:

$$\tau_m \frac{dm}{dt} \approx 0 \quad \Rightarrow \quad m \approx m_\infty(u)$$

- **Slow combined recovery variable (w).** The other two gating variables, sodium inactivation h and potassium activation n , evolve more slowly and on similar timescales (see Figure 4). Moreover, the quantity $1 - h(t)$ behaves similarly to $n(t)$: both increase with depolarization and act as slow recovery processes that oppose changes in voltage. For this reason, many reduced models combine their effects into a single slow variable w , called the **recovery variable**.

With these approximations, the system reduces to two equations:

$$C \frac{du}{dt} = F(u, w) + I, \quad \tau_w \frac{dw}{dt} = G(u, w)$$

We can visualize this simplified system in the **phase plane**, a 2D plot where the horizontal axis is the voltage u and the vertical axis is the recovery variable w . In this plane we draw:

- **Nullclines:** curves where $du/dt = 0$ (the u -nullcline) or $dw/dt = 0$ (the w -nullcline). On these curves, the corresponding **variable does not change**. The point where the nullclines cross is called a **fixed point** (point where the two derivatives are zero). It represents a possible steady state of the neuron (a state in which the membrane voltage no longer changes).
- **Vector field:** at each point (u, w) we draw a small arrow that shows how the system would move if it were sitting at that point. The arrow points in the **direction of change** (set by the signs of $\frac{du}{dt}$ and $\frac{dw}{dt}$). Its length (Δt) shows how quickly the system moves there: a long arrow means the system changes quickly, while a short arrow means it changes slowly.

$$\begin{bmatrix} \Delta u \\ \Delta w \end{bmatrix} = \Delta t \begin{bmatrix} \frac{du}{dt} \\ \frac{dw}{dt} \end{bmatrix}$$

- **Trajectories:** curves that follow the vector field and show how the system evolves over time from different initial conditions.

5.2 Stability: Graphical intuition

A quick way to check the stability of a fixed point is to look at the vector field in the phase plane. Each small arrow shows the direction in which the system moves for a given pair (u, w) .

- **On the u -nullcline** ($du/dt = 0$), the voltage does not change. The motion can only be up or down in the w direction, so the arrows are **vertical**.
- **On the w -nullcline** ($dw/dt = 0$), the recovery variable does not change. The motion can only be left or right in the u direction, so the arrows are **horizontal**.
- When you cross a nullcline, the sign of that derivative changes (e.g., crossing $du/dt = 0$ switches voltage from increasing to decreasing, or vice-versa).
- Away from both nullclines, both u and w change at the same time, so the arrows are **diagonal**.

If arrows around a fixed point point toward it, the point is **stable**: small perturbations decay and the system returns to that state. If arrows point away, the point is **unstable**: small perturbations grow and the system moves away.

5.3 Stability: Mathematical view

The graphical arrow method is intuitive, but we can also check stability more formally using mathematics. The idea is simple:

1. **Near a fixed point, the system behaves almost linearly.** We ask: if we make a tiny perturbation $(\Delta u, \Delta w)$, does it **grow or shrink** with time?
2. To answer this, we build the **Jacobian matrix**, which summarizes how the system changes locally:

$$J = \begin{bmatrix} \frac{\partial F}{\partial u} & \frac{\partial F}{\partial w} \\ \frac{\partial G}{\partial u} & \frac{\partial G}{\partial w} \end{bmatrix}_{(u_{fp}, w_{fp})}$$

Each entry measures how sensitive one variable's rate of change is to small changes in u or w . In other words, the Jacobian is the **linear approximation** of the nonlinear system near the fixed point.

3. The **eigenvalues** λ_1, λ_2 of J describe how small perturbations evolve near the fixed point. They are defined by the characteristic equation:

$$\det(J - \lambda I) = \lambda^2 - (F_u + G_w)\lambda + (F_u G_w - F_w G_u) = 0$$

Each eigenvalue tells us whether small deviations **grow**, **decay**, or **oscillate** in time, and how fast.

4. The relation between the eigenvalues and the Jacobian entries is:

$$\lambda_1 + \lambda_2 = \text{Tr}(J) = F_u + G_w, \quad \lambda_1 \lambda_2 = \det(J) = F_u G_w - F_w G_u$$

5. Stability then follows directly:

- If both eigenvalues have negative real parts, perturbations decay $\Rightarrow$ the fixed point is **stable**.
- If at least one eigenvalue has a positive real part, perturbations grow $\Rightarrow$ the fixed point is **unstable**.
- If eigenvalues are complex, trajectories spiral in or out depending on the sign of their real part.

6. **Using the trace and determinant, we can state the stability rule compactly:**

$$\text{Stable fixed point: } \text{Tr}(J) < 0 \quad \text{and} \quad \det(J) > 0$$

The trace tells us whether changes in the system grow or decay, and the determinant tells us if both variables (u and w) change in the same way or in opposite ways. When the trace is negative and the determinant is positive, both variables return toward the fixed point, the system is stable.

This formal test is exactly the mathematical version of the arrow picture: **negative trace means the system contracts overall, and positive determinant means both directions work together to pull trajectories inward.**

6 Simplified models of neuronal excitability

Detailed models such as Hodgkin–Huxley describe action potential generation with great biological accuracy, but involve four variables and many parameters. Simplified, low-dimensional models reproduce the essential features of neuronal excitability using only one or two equations. Two important examples are the **FitzHugh–Nagumo** and **Morris–Lecar** models.

6.1 The FitzHugh–Nagumo model

The FitzHugh–Nagumo (FHN) model [3, 4] is a simple, two-variable system that captures the main features of excitability. It is not a biological reduction of the Hodgkin–Huxley model but a **mathematical caricature** that preserves its core dynamical structure. The model has:

- **Fast variable** u (voltage-like), which produces threshold and the rapid up–down motion of a spike.
- **Slow variable** w (recovery), which provides delayed negative feedback and drives the system back to rest.

The equations are:

$$\begin{cases} \frac{du}{dt} = u - \frac{1}{3}u^3 - w + I \\ \frac{dw}{dt} = \varepsilon(bu - w) \end{cases} \quad (7)$$

where I is the input current, ε sets the timescale separation, and b controls how strongly w follows changes in u . The interaction between fast excitation (u) and slow recovery (w) produces the characteristic spike–recovery cycle.

Nullclines, fixed points, and the vector field

To understand the system, we examine the **nullclines**, the curves where each variable stops changing:

- **u -nullcline** ($du/dt = 0$):

$$w = u - \frac{1}{3}u^3 + I$$

This is a cubic, S-shaped curve showing where u remains constant.

- **w -nullcline** ($dw/dt = 0$):

$$w = bu$$

This is a straight line through the origin showing where w remains constant.

The intersections of the nullclines are the fixed points (u_{fp}, w_{fp}).

Away from the nullclines, the system moves according to the derivatives. For a small time step Δt :

$$\begin{bmatrix} \Delta u \\ \Delta w \end{bmatrix} = \Delta t \begin{bmatrix} \frac{du}{dt} \\ \frac{dw}{dt} \end{bmatrix}$$

This tells us:

- **Sign of $\frac{du}{dt}$** : determines whether u increases (arrow points right) or decreases (arrow points left).
- **Sign of $\frac{dw}{dt}$** : determines whether w increases (arrow points up) or decreases (arrow points down).
- **Magnitude of the derivatives**: determines arrow length, with larger derivatives producing longer arrows.

Plotting these arrows across the plane yields the **vector field**. Then, trajectories follow these arrows over time.

Example cases

Let us evaluate the dynamics at the point $(u, w) = (0, 1)$ for the parameters $b_0 = 2$ and $b_1 = 2$. The system is:

$$\frac{du}{dt} = u - \frac{1}{3}u^3 - w, \quad \frac{dw}{dt} = b_0 + b_1u - w$$

Substituting $u = 0$, $w = 1$, $b_0 = 2$, and $b_1 = 2$ gives:

$$\frac{du}{dt} = -1, \quad \frac{dw}{dt} = 1$$

During a small time step Δt :

$$\begin{bmatrix} \Delta u \\ \Delta w \end{bmatrix} = \Delta t \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Interpretation (Figure 5):

- u **decreases**: arrow points **left**.
- w **increases**: arrow points **up**.

The same procedure applies to any point in the phase plane: the signs set the direction and the magnitudes set the arrow length.

Example 1: the case $I = 0$

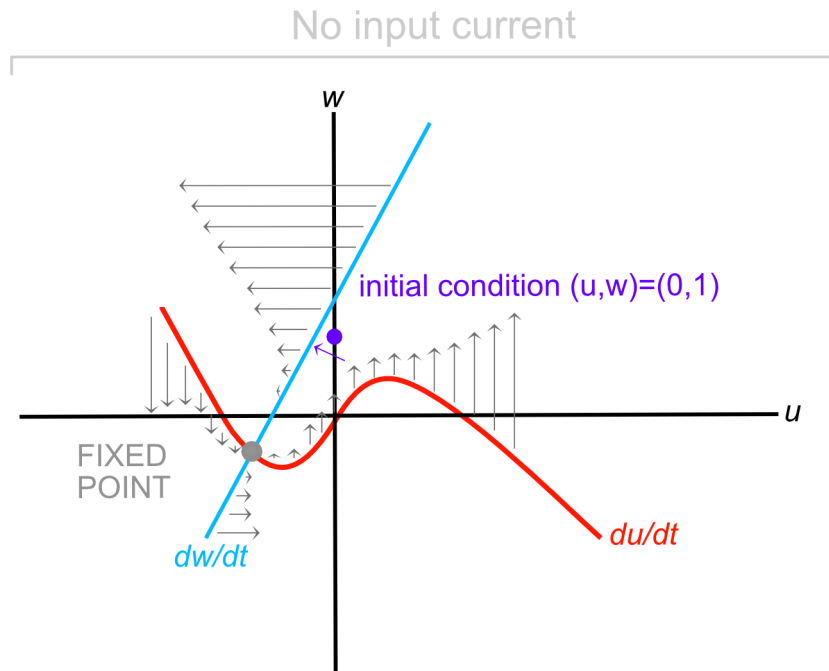


Figure 5: Phase-plane analysis of the FitzHugh–Nagumo model for $I = 0$

Example 2: A brief current pulse

Now we examine what happens when the neuron receives a very short current pulse. Let the input be:

$$I(t) = q \delta(t)$$

a brief impulse of amplitude q delivered at $t = 0$. Because this pulse acts only through the voltage equation, it causes an **instantaneous jump in u** , while the recovery variable w is unchanged (it does not have time to respond).

In the phase plane, this shows up as a **horizontal jump**. Whatever the current state of the system is at the moment of the pulse (often the fixed point if the neuron is at rest) it suddenly shifts left or right depending on the sign of q . Only the u -coordinate changes; w stays fixed.

To see this more concretely, suppose the system starts at the fixed point (gray dot in Figure 6). A positive brief pulse “kicks” the state horizontally to the green point. After this momentary jump, the input returns to $I = 0$, so the system once again follows the same phase-plane dynamics as in the no-input case. From the new starting point, its motion is described by:

$$\begin{bmatrix} \Delta u \\ \Delta w \end{bmatrix} = \Delta t \begin{bmatrix} \frac{du}{dt} \\ \frac{dw}{dt} \end{bmatrix}$$

exactly as before. The signs of these derivatives determine which direction the trajectory moves next, and the magnitudes determine how fast it moves (how long the arrows are).

In summary:

- **A brief pulse produces a horizontal jump in the phase plane.**
- **After the jump, the trajectory evolves under the same $I = 0$ vector field.**
- **Whether the pulse triggers a spike depends on which side of the cubic nullcline the jump lands on.**

Thus, **a short current pulse does not change the rules of the system.** It only changes where the system starts its motion after the pulse, and that starting point determines whether the neuron emits a spike.

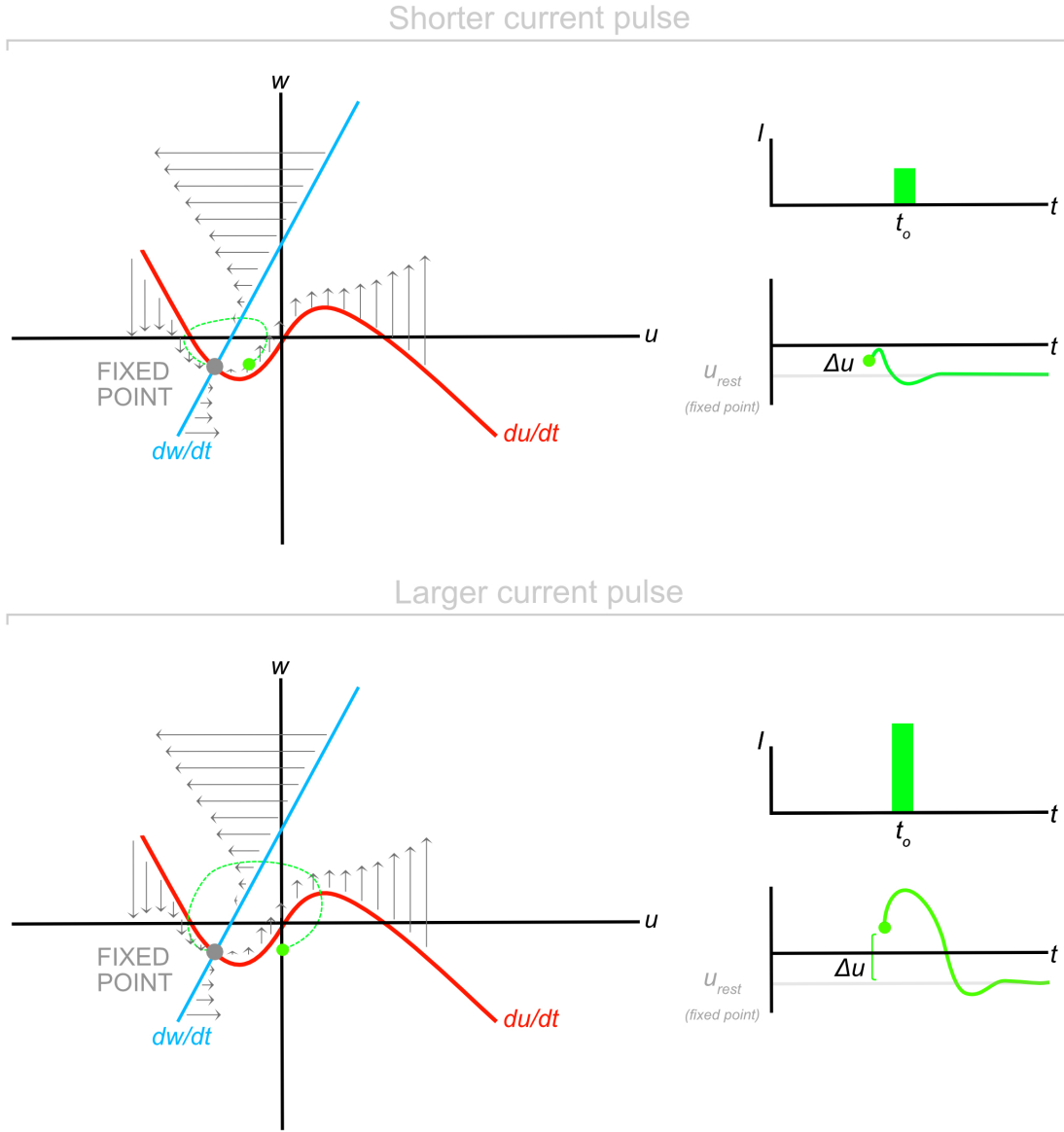


Figure 6: Phase-plane analysis of the FitzHugh–Nagumo model under brief current pulses. A short pulse produces a small horizontal jump in u and the trajectory relaxes back to rest. A larger pulse produces a bigger jump that crosses the cubic nullcline, before returning to the fixed point.

Example 3: Constant input (Emergence of oscillations)

So far we have analyzed the dynamics for $I = 0$ and for a brief pulse. We now study what happens when the neuron receives a **constant input current**:

$$I(t) = I_0$$

A constant input modifies the u -equation, which means the u -nullcline shifts vertically:

$$w = u - \frac{1}{3}u^3 + I_0$$

For $I_0 > 0$, the entire cubic moves upward. As a result, the fixed point also moves. Depending on the value of I_0 , the fixed point may be:

- Stable (the system returns to rest).
- Unstable (the system is pushed away from rest).

When the fixed point becomes unstable, trajectories no longer converge to rest but instead approach a **limit cycle** (green dashed curve in Figure 7). This corresponds to a sustained oscillation in $u(t)$, representing repetitive spiking.

How constant current generates a limit cycle As the cubic nullcline shifts upward:

- The fixed point approaches the middle branch of the cubic.
- Eventually it enters the unstable region.
- The system is repelled from the fixed point and settles onto a stable periodic orbit.

In the voltage trace on the right, the neuron emits continuous spikes for as long as I_0 is applied. When the input stops, the system returns to its resting state.

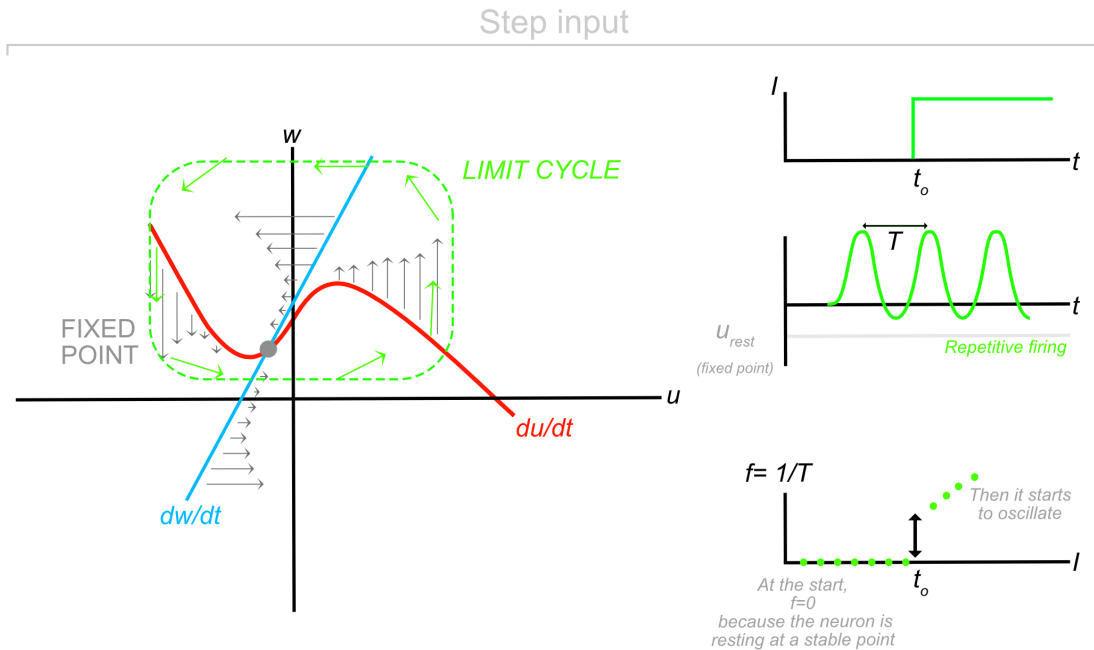


Figure 7: Phase-plane analysis of the FitzHugh–Nagumo model for a step input. When the current switches on at t_0 , the u -nullcline shifts upward and the fixed point becomes unstable. The trajectory is then attracted to a stable limit cycle (green dashed curve), producing repetitive firing with period T . Before the current step, the system sits at the stable fixed point and has firing rate $f = 0$.

Type II excitability

The FitzHugh–Nagumo model exhibits **Type II excitability**. This means that, as the input current I_0 is increased, the neuron does not transition gradually into spiking. Instead, the **onset of rhythmic firing occurs abruptly**. In dynamical-systems terms, this transition is characterized by:

- **Loss of stability of the resting state:** The fixed point corresponding to rest becomes unstable, so small perturbations no longer decay but instead grow over time.
- **Immediate emergence of a limit cycle:** Once the fixed point loses stability, the system jumps directly onto a stable, closed orbit in phase space.
- **Finite-frequency firing at onset:** Because the limit cycle has a well-defined period, spiking begins with a non-zero firing rate; the frequency cannot approach zero.

In simple terms, Type II neurons display an **abrupt onset of spiking**: as soon as they begin to fire, they do so with a clear and finite rhythm. The FitzHugh–Nagumo model consistently shows this behavior, which is why it is classified as a Type II excitability model.

6.2 The Morris–Lecar model

The Morris–Lecar (ML) model [5] is a two-variable model of neuronal excitability that retains a clear biophysical interpretation while remaining low dimensional. By eliminating fast sodium activation while retaining slow potassium dynamics, it preserves realistic nullcline geometry and excitability properties while remaining analytically tractable.

Compared to the FitzHugh–Nagumo model, it is therefore **more biophysically grounded**, as its nonlinearities arise directly from voltage-dependent ion channels.

The model describes the membrane potential u and a single recovery variable w representing potassium-channel activation:

$$\begin{aligned} C \frac{du}{dt} &= -g_{Ca} m_{\infty}(u) (u - E_{Ca}) - g_K w (u - E_K) - g_L (u - E_L) + I \\ \frac{dw}{dt} &= \frac{w_{\infty}(u) - w}{\tau_w(u)} \end{aligned} \quad (8)$$

Here, u denotes the membrane potential and w is a slow recovery variable. Calcium activation is assumed to be fast and is therefore replaced by its instantaneous steady-state value $m_{\infty}(u)$, while potassium activation evolves dynamically through w .

The voltage-dependent steady-state functions and time constant are given by:

$$m_{\infty}(u) = \frac{1}{2} \left[1 + \tanh \left(\frac{u - u_1}{u_2} \right) \right], w_{\infty}(u) = \frac{1}{2} \left[1 + \tanh \left(\frac{u - u_3}{u_4} \right) \right], \tau_w(u) = \frac{T_w}{\cosh \left(\frac{u - u_3}{2u_4} \right)}$$

The sigmoidal shapes of $m_{\infty}(u)$ and $w_{\infty}(u)$ reflect the probabilistic opening of calcium and potassium channels. The parameters u_1 and u_3 set the voltages at which these channels are half-activated, while u_2 and u_4 determine how steeply activation changes with voltage.

Figure 8 illustrates the resulting nullclines in the phase plane. Their intersections correspond to the fixed points of the system. Depending on parameter values and input current, different types of fixed points may arise (a full stability analysis can be carried out using the methods introduced in the previous section):

- **Stable fixed point:** If the system starts close to this point, it moves toward it over time. Small disturbances fade away, and the neuron returns to its resting state.
- **Unstable fixed point:** Small disturbances grow, causing the system to move away from the fixed point and potentially leading to oscillatory activity.
- **Saddle fixed point:** The system is attracted to this point from some directions but repelled from others. Such points often lie at the boundary between small, subthreshold responses and large excursions corresponding to spikes.

Type I excitability in Morris–Lecar

The nullclines intersect so that the stable fixed point lies very close to a saddle fixed point. This creates a region of **very slow flow** near the fixed points, visible in the figure as short arrows along the left branch of the u -nullcline. To illustrate this effect, Figure 8 shows the response to three brief input pulses of increasing amplitude:

- **Purple pulse:** too weak to push the system out of the resting region. The trajectory is nudged slightly but simply relaxes back to the stable fixed point.
- **Blue pulse:** strong enough to push the system just past the boundary between “resting” and “spiking” behavior, but it lands very close to the saddle. Because the flow is extremely slow near the saddle, the trajectory *hangs around* this region before finally escaping and producing a spike. This creates a **delayed first spike**.
- **Orange pulse:** pushes the system far past the slow region. The trajectory moves away quickly and produces a spike with almost no delay.

Responses to different current pulses

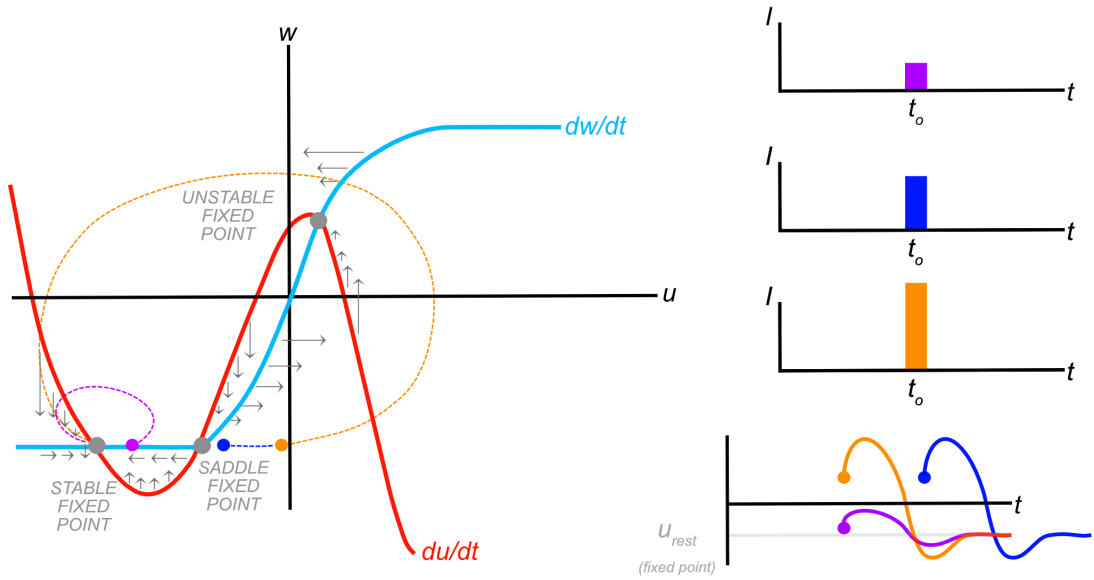


Figure 8: Responses of the Morris–Lecar model to current pulses of increasing amplitude. The stable and saddle fixed points create a region of slow dynamics near threshold. A weak pulse decays back to rest (purple), a near-threshold pulse lingers near the saddle and produces a delayed spike (blue), and a larger pulse escapes quickly and generates a fast spike (orange). This slow passage near the saddle underlies Type I excitability.

This slow motion near the rest state explains the defining feature of Type I excitability:

- Near threshold, the system spends a long time near the fixed point.
- This produces **arbitrarily long interspike intervals**.
- Which means the **firing frequency can start arbitrarily close to zero**.

In other words, in Type I neurons the onset of spiking is **gradual**. The dynamical slowing near the saddle-node pair is what creates the characteristic low-frequency firing at threshold.

From pulses to constant input

The Type I behavior seen with brief pulses also appears when the neuron is driven by a **constant input** I_0 . In this case, the u -nullcline shifts upward (Figure 9). Once I_0 reaches a critical value, the u -nullcline and w -nullcline no longer intersect: the stable and saddle fixed points **collide and disappear**. When this happens:

- There is no resting state for the system to settle into.
- The trajectory is forced to circulate around the phase plane.
- But it still moves very slowly through the region where the fixed point used to be.

This slow passage makes each cycle take a long time, so the neuron fires at a **very low frequency** right above threshold. As I_0 increases, the slow region is crossed more quickly, and the firing rate increases smoothly.

Thus, constant current produces the same signature of Type I excitability that we saw with pulses: **the onset of firing is gradual, with arbitrarily low frequency at threshold**.

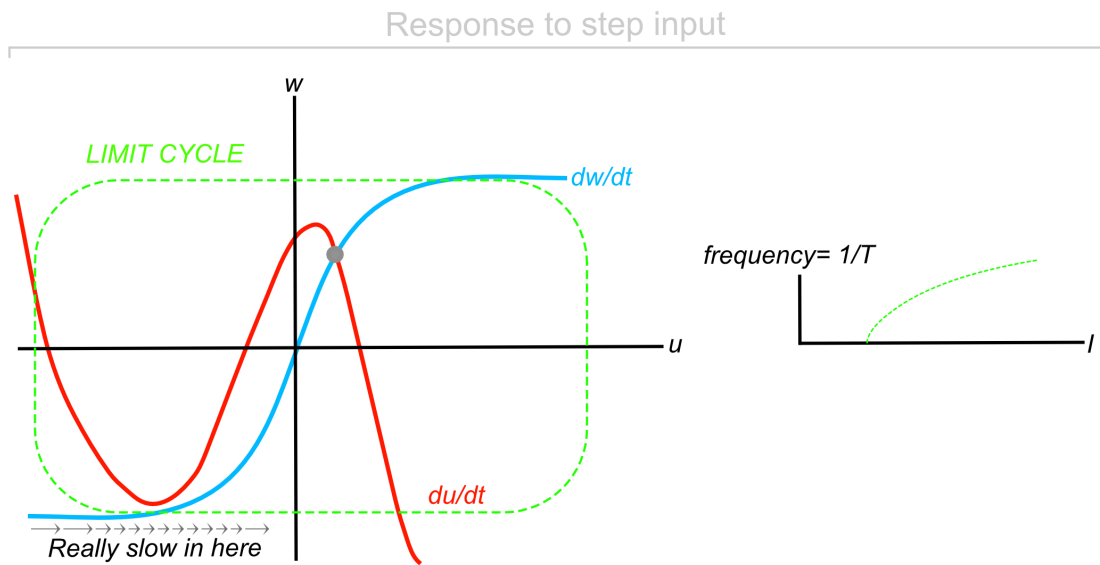


Figure 9: Phase-plane analysis of the Morris–Lecar model under a constant step current. When the current is strong enough to eliminate the resting fixed point, the trajectory is forced onto a limit cycle. Just above threshold, it passes slowly through the region where the fixed point used to be, producing low-frequency oscillations typical of Type I excitability.

7 Generalized integrate-and-fire models

7.1 From LIF to nonlinear models

In a previous chapter, we saw that the LIF model is only one example of a broader family of systems described by:

$$\tau \frac{du}{dt} = F(u) + RI$$

together with a threshold–reset rule. We also introduced several possible choices for the function $F(u)$, each of which defines a different integrate-and-fire (IF) model (Figure 10):

- **LIF** (a linear leak term):

$$F(u) = -(u - u_{\text{rest}})$$

- **Quadratic IF (QIF)** (a parabola):

$$F(u) = c_2(u - c_1)^2 + c_0$$

which captures the normal form of a saddle–node bifurcation and provides a simple Type I spiking model.

- **Exponential IF (EIF)** (an exponential rise near the threshold):

$$F(u) = -(u - u_{\text{rest}}) + c_0 e^{u-\theta}$$

producing a soft, more realistic spike initiation.

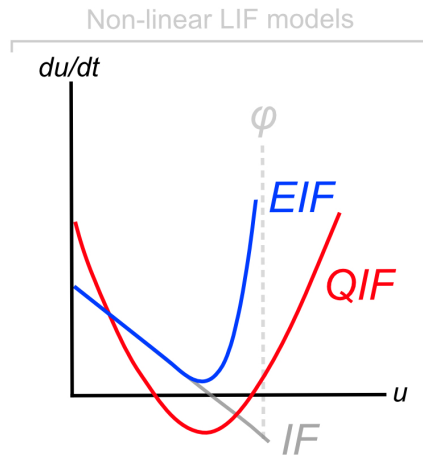


Figure 10: Subthreshold voltage dynamics $F(u)$ in different nonlinear LIF models. The classic LIF model (grey) uses a linear leak term. The quadratic IF model (QIF, red) introduces a parabolic nonlinearity, producing a smooth onset of spiking. The exponential IF model (EIF, blue) includes an exponential rise near threshold, mimicking the sharp spike initiation observed in many real neurons.

Despite their different shapes, all these **models rely on the same basic architecture**: a stable resting potential u_{rest} that attracts the voltage when there is no input, and a threshold–reset rule that generates spikes when $u(t)$ becomes too large. **They differ only in the choice of the subthreshold function $F(u)$, which controls how the voltage approaches threshold. Different forms of $F(u)$ are used because real neurons have different spike-initiation dynamics**: some depolarize smoothly (well approximated by a parabola), while others show a sharp exponential takeoff near threshold. Choosing $F(u)$ appropriately allows an integrate-and-fire model to capture these distinct biophysical behaviors while keeping the model simple.

7.2 Adaptive integrate-and-fire models

Real neurons often show **spike-triggered adaptation**: **after a spike, it becomes temporarily harder to spike again**. A one-variable IF model cannot reproduce this, because the voltage

alone has no memory of past spikes. To capture this effect, we introduce a second variable w that represents an **adaptation current**. After each spike:

$$w \rightarrow w + b$$

Here, b is a constant that determines how much the adaptation current increases after a spike. A larger b means a stronger adaptive effect, making the neuron noticeably less excitable for a short period of time. Including this adaptive variable allows the model to reproduce several biologically important behaviors, such as gradually slowing down during sustained firing or producing brief bursts of spikes, that simple LIF-type models cannot generate. Two widely used adaptive IF models that implement this idea are the Izhikevich model and the adaptive exponential IF (AdEx) model.

Quadratic adaptive integrate-and-fire model (Izhikevich-type)

The quadratic adaptive integrate-and-fire model, inspired by the work of Izhikevich [6], extends a quadratic voltage model by including a slow adaptation variable:

$$\begin{cases} \tau_m \frac{du}{dt} = (u - u_{\text{rest}})(u - \varphi) - Rw + RI \\ \tau_w \frac{dw}{dt} = a(u - u_{\text{rest}}) - w \end{cases} \quad (9)$$

Spiking is implemented through a **threshold-and-reset rule** (when the membrane potential reaches the threshold φ , it is reset to the resting value and the adaptation variable is increased):

$$u = \varphi \Rightarrow \begin{cases} u \rightarrow u_{\text{rest}} \\ w \rightarrow w + b \end{cases}$$

The increase in w makes the term $-Rw$ larger, acting as a stronger negative current that temporarily reduces the neuron's excitability. In this model:

- The membrane potential u **follows a quadratic integrate-and-fire dynamics**, leading to rapid divergence once threshold is approached.
- The variable w represents an **adaptation current** that increases when the membrane potential rises (set by a) and jumps up after each spike (set by b), **reducing the neuron's excitability**.

By tuning its parameters, this model can reproduce tonic firing, spike-frequency adaptation, bursting, and resonant firing patterns.

Adaptive exponential integrate-and-fire model (AdEx)

The adaptive exponential integrate-and-fire (AdEx) model [7] combines:

- The linear leak current of the LIF model.
- An exponential term capturing spike initiation.
- A slow adaptation current w .

Its subthreshold dynamics are given by:

$$\begin{cases} \tau_m \frac{du}{dt} = -(u - u_{\text{rest}}) + \Delta_T \exp\left(\frac{u - \varphi}{\Delta_T}\right) - Rw + RI(t) \\ \tau_w \frac{dw}{dt} = a(u - u_{\text{rest}}) - w \end{cases} \quad (10)$$

Spikes are generated using the same reset mechanism:

$$u = \varphi \Rightarrow \begin{cases} u \rightarrow u_{\text{rest}} \\ w \rightarrow w + b \end{cases}$$

The AdEx model is highly flexible and can reproduce a wide range of firing behaviors observed in cortical neurons, including regular spiking, fast spiking, bursting, and spike-frequency adaptation.

8 The Cable Equation

All the neuron models we have used so far (LIF, QIF, EIF, Izhikevich, AdEx, FitzHugh–Nagumo, and Morris–Lecar) treat the neuron as a **point neuron**. A point neuron treats the entire cell as a **single electrical compartment**, so the **membrane potential depends only on time**, $u(t)$, and all inputs are assumed to act at one location. This simplification is useful for studying excitability and spiking, but **real neurons have spatial structure that cannot be captured by such a model**.

Why point-neuron models are not enough

Neurons have a rich **spatial structure**:

- Dendrites receiving inputs at different locations.
- A soma integrating the signals.
- An axon carrying the output spike.

Because these structures are long and thin, the membrane potential is **not the same everywhere**. A synaptic input arriving on a distal dendrite produces a different voltage change at the soma than an input arriving nearby, because voltage spreads and decays as it travels.

Thus, to understand how neurons combine many synaptic inputs distributed across their dendritic tree, we must describe **voltage as a function of both space and time**:

$$u = u(x, t)$$

8.1 Deriving the cable equation

To understand how voltage spreads along a dendrite, we model the dendrite as a long **electrical cable** made of many small segments of length Δx (Figure 11). Each tiny segment behaves like a simple circuit:

- **Along the dendrite**: current flows through the cytoplasm, which has resistance.
- **Across the membrane**: current can leak through ion channels or charge the membrane capacitance.

The idea is simple: **current flows along the cable, some of it leaves through the membrane, and the rest continues forward**.

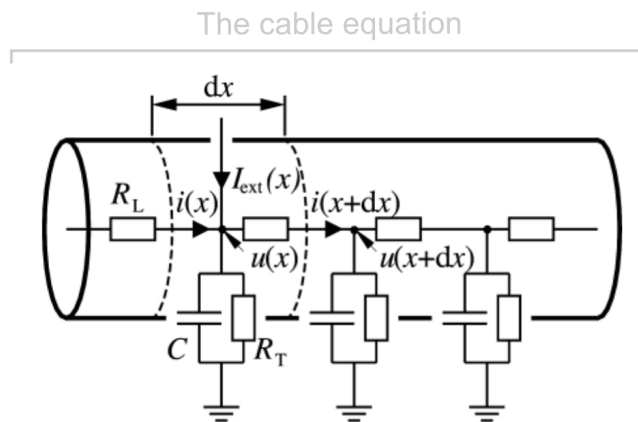


Figure 11: Schematic of a dendritic segment used to derive the cable equation. Adapted from Fig. 3.5 in Reference [8].

Let $u(x, t)$ be the membrane potential and $i(x, t)$ the current that flows along the inside of the dendrite. For notational simplicity, we write ordinary derivatives d/dx and d/dt instead of partial derivatives, keeping in mind that $u(x, t)$ depends on both space and time. With these definitions in place, we can now derive the cable equation:

1. Ohm's law along the dendrite Because the inside of the dendrite has electrical resistance, there is a small voltage drop between two neighboring segments:

$$u(x, t) - u(x + \Delta x, t) = R_i \Delta x i(x, t)$$

To move from this discrete expression to a continuous one (treating the dendrite as a smooth cable rather than a chain of small segments), we divide both sides by the segment length Δx :

$$\frac{u(x, t) - u(x + \Delta x, t)}{\Delta x} = R_i i(x, t)$$

Dividing by Δx gives the change in voltage per unit length. When we now let $\Delta x \rightarrow 0$, this finite difference becomes a spatial derivative:

$$\frac{du}{dx} = -R_i i(x, t)$$

In other words, **making the segments arbitrarily small turns the dendrite into a continuous cable**, and the voltage drop along it is described by a derivative instead of a difference between two points. Differentiating once more:

$$\frac{d^2 u}{dx^2} = -R_i \frac{di}{dx}$$

This links the curvature of the voltage profile (the second spatial derivative of u) to how the axial current changes along the cable.

2. Current conservation Current conservation simply means that **current cannot disappear or accumulate inside the dendrite**. So the difference between current entering and leaving a small segment must equal the current crossing the membrane:

$$i(x, t) - i(x + \Delta x, t) = \Delta x \left[C_m \frac{du}{dt} + \sum_{\text{ion}} i_{\text{ion}}(x, t) - i_{\text{ext}}(x, t) \right]$$

Here:

- $C_m du/dt$ is the capacitive current.
- $\sum i_{\text{ion}}$ is the total ionic current.
- i_{ext} is externally injected current.

Dividing by Δx and letting $\Delta x \rightarrow 0$:

$$-\frac{di}{dx} = C_m \frac{du}{dt} + \sum_{\text{ion}} i_{\text{ion}} - i_{\text{ext}}$$

3. Putting both pieces together From Ohm's law:

$$\frac{d^2 u}{dx^2} = -R_i \frac{di}{dx}$$

From current conservation we substitute $\partial i / \partial x$ and obtain:

$$\boxed{\frac{d^2 u(x, t)}{dx^2} = R_i C_m \frac{du}{dt} + R_i \sum_{\text{ion}} i_{\text{ion}}(x, t) - R_i i_{\text{ext}}(x, t)} \quad (11)$$

This final expression is the **cable equation**. It describes how **voltage spreads along a dendrite** and how input at **one location affects the membrane potential at other points** on the cell.

9 Model of a Synapse

9.1 Modeling synapses: Conductances and postsynaptic currents

A synapse is the junction through which one neuron communicates with another. It connects a **presynaptic** neuron, which sends the signal, to a **postsynaptic** neuron, which receives it. When an action potential reaches the presynaptic terminal, a sequence of steps occurs:

1. Voltage-gated Ca^{2+} channels open, allowing calcium ions to enter the terminal.
2. The influx of calcium triggers the release of neurotransmitter molecules into the synaptic cleft.
3. These neurotransmitters bind to receptor-gated ion channels on the postsynaptic membrane.
4. The channels open, ions flow across the membrane, and the postsynaptic potential changes.

Just as the Hodgkin–Huxley model describes voltage-gated channels, we can describe **transmitter-gated channels** that open when neurotransmitters bind to their receptors. This binding changes the membrane conductance, **allowing ions to flow and generate a postsynaptic current**:

$$I_{\text{syn}}(t) = g_{\text{syn}}(t) [u(t) - E_{\text{syn}}] \quad (12)$$

This expression shows that:

- $g_{\text{syn}}(t)$ is the **synaptic conductance**, describing how strongly the synapse is open at time t .
- E_{syn} is the **reversal potential**, the membrane voltage at which the synaptic current is zero.

When the presynaptic neuron emits a spike at time t_j , the conductance briefly increases and then decays back to zero. A simple and widely used model describes this process as an exponential decay:

$$g_{\text{syn}}(t) = \sum_{t_j} \bar{g}_{\text{syn}} e^{-\frac{(t-t_j)}{\tau_{\text{syn}}}} \Theta(t-t_j) \quad (13)$$

- $\bar{g}_{\text{syn}}$: maximum conductance reached after a spike.
- τ_{syn} : synaptic time constant (how fast the effect decays).
- $\Theta(t-t_j)$: **Heaviside step function**. It equals 0 before the spike ($t < t_j$) and 1 after the spike ($t \geq t_j$), ensuring that the **exponential contribution starts only once the spike has occurred**.

Each presynaptic spike therefore adds a short, transient increase in conductance. If another spike arrives before the previous effect has vanished, the responses add up. In this way, $g_{\text{syn}}(t)$ acts as a **low-pass filter**, transforming brief presynaptic spikes into smooth conductance changes, while $I_{\text{syn}}(t)$ converts these changes into postsynaptic voltage responses depending on E_{syn} .

9.2 Differences between excitatory and inhibitory synapses

Synapses can either **excite** or **inhibit** the postsynaptic neuron. Their effect depends on the **reversal potential** E_{syn} , which determines the direction of ion flow when the synapse opens.

- **Excitatory synapses:** If $E_{\text{syn}} > u_{\text{rest}}$, ions flow so that the membrane becomes more positive (**depolarization**). This brings the neuron closer to firing threshold and increases the chance of a spike. Most excitatory synapses use the neurotransmitter glutamate, with $E_{\text{syn}} \approx 0$ mV. The resulting voltage change is an **excitatory postsynaptic potential (EPSP)**.
- **Inhibitory synapses:** If $E_{\text{syn}} < u_{\text{rest}}$, the synapse drives the membrane potential downward (**hyperpolarization**), making a spike less likely. Most inhibitory synapses use the neurotransmitter GABA, with $E_{\text{syn}} \approx -75$ mV. The resulting voltage change is an **inhibitory postsynaptic potential (IPSP)**.

In short, the sign of $(E_{\text{syn}} - V_{\text{rest}})$ determines whether a synapse is excitatory or inhibitory. Each neuron continuously combines many excitatory and inhibitory inputs, and their **dynamic balance** shapes how information flows through the network.

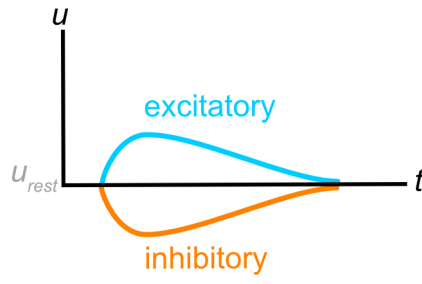


Figure 12: Voltage response to a brief excitatory (blue) or inhibitory (orange) synaptic input. Excitatory input depolarizes the membrane above rest, whereas inhibitory input hyperpolarizes it below u_{rest} .

10 Variability and Noise in Neural Activity

Neuronal activity recorded *in vivo* is highly variable: **spike times fluctuate from trial to trial**, and even **nearby neurons often show only weak correlations**. This raises an important question, does this variability simply reflect “noise,” or is it part of how neurons encode information? Several intrinsic factors contribute to this variability. Thermal fluctuations introduce tiny random changes in the membrane potential, and ion channels open and close stochastically rather than deterministically. Even though these effects are small, together they produce **noticeable variability in the responses of real neurons**. Noise can be introduced mathematically at several levels of a neuron model. Each approach captures a different biological source of variability.

10.1 Noise in the inputs

We can add a random term to the input current:

$$\tau_m \frac{du}{dt} = F(u) + RI^{\text{det}} + RI^{\text{noise}} \quad (14)$$

Here, I^{det} is the **deterministic** (predictable) part of the input, while I^{noise} represents **random fluctuations** around it. If these fluctuations change very rapidly in time, they are called **white noise**; if they vary more smoothly, they are called **colored noise**. Colored noise can be modeled as white noise passed through a low-pass filter, mimicking how real synaptic inputs are smoothed by the membrane.

Here, $F(u)$ is a general function describing the intrinsic membrane dynamics. Different choices of $F(u)$ recover the various integrate-and-fire models introduced above, such as the LIF, quadratic IF, and AdEx models.

10.2 Stochastic spike arrival

Instead of continuous noise, we can model inputs as random presynaptic spikes:

$$\tau_m \frac{du}{dt} = F(u) + RI^{\text{det,ext}} + \sum_k w_k \delta(t - t_k^{(f)}) \quad (15)$$

Each term $\delta(t - t_k^{(f)})$ represents a spike arriving at time $t_k^{(f)}$ with synaptic weight w_k . Because the spike times are irregular, the postsynaptic potential $u(t)$ naturally fluctuates.

10.3 Noise in the output

Finally, a neuron’s own spiking can be treated probabilistically. Instead of firing at a fixed threshold, we can assign a firing **probability** that depends on the membrane potential:

$$s = F(u(t) - \varphi)$$

Here, F is a function that maps voltage to the probability of spiking. **The closer the potential $u(t)$ is to the threshold φ , the higher the probability of generating a spike**. This captures the idea that neurons are not perfect switches, tiny variations in inputs or channel states can make them fire earlier or later.

In reality, all these sources of variability interact to produce the irregular firing patterns seen in the brain. Far from being purely “damaging”, such noise can **enhance sensitivity**, **extend dynamic range**, and even **improve information coding** by helping neurons explore a wider range of responses.

11 Characterizing Neural Activity

Type of measure	How to compute it	Purpose / Interpretation
Spike rate of a single neuron	Count the total number of spikes n_{sp} emitted within a time window T	Average firing rate: $r = \frac{n_{\text{sp}}}{T}$
<i>→ Fano factor (variability across trials = how much the spike count fluctuates from trial to trial)</i>	Repeat the same measurement over many trials and, within a fixed time window, compute the mean and variance of spike counts across trials.	Quantifies trial-to-trial variability in spike counts: $F = \frac{\text{variance}}{\text{mean}} = \frac{\text{Var}(n_{\text{sp}})}{\langle n_{\text{sp}} \rangle}$ If $F = 1$, firing is Poisson-like (random, independent spikes); $F < 1$, firing is more regular (spikes occur almost periodically); $F > 1$, firing shows extra variability or bursts.
Spike density of a single neuron with peri-stimulus time histogram (PSTH)	Align multiple trials to the same stimulus onset. For each small time bin of width Δt , sum the number of spikes across all K repetitions, then divide by $K\Delta t$.	Provides a time-resolved estimate of firing rate: $\rho(t) = \frac{1}{K\Delta t} \sum_{k=1}^K n_k(t, t + \Delta t)$ Reveals how the firing rate evolves relative to a repeated event or stimulus.
Interspike interval (ISI)	Measure the time intervals between consecutive spikes $s_k = t_{k+1} - t_k$ and estimate their distribution $P_0(s)$. The firing rate is the inverse of the mean ISI.	Characterizes temporal regularity: $\langle s \rangle = \int_0^\infty s P_0(s) ds, \quad C_V = \frac{\sqrt{\langle s^2 \rangle - \langle s \rangle^2}}{\langle s \rangle}$ Coefficient of variation: $C_V = 1 \rightarrow$ Poisson-like (random); $C_V < 1 \rightarrow$ regular, nearly periodic firing; $C_V > 1 \rightarrow$ bursting or irregular activity. <i>Note: the Fano factor measures variability across trials, while C_V measures irregularity within a spike train.</i>
Population rate (many neurons)	Average the activity across a group of N neurons. Count the total number of spikes emitted by all neurons in a small time bin $[t, t + \Delta t]$.	Describes collective population activity: $A(t) = \frac{1}{N\Delta t} n_{\text{act}}(t, t + \Delta t)$ where $n_{\text{act}}(t, t + \Delta t)$ is the total number of spikes fired by the population in that interval. <i>Equivalent continuous form:</i> $A(t) = \frac{1}{N\Delta t} \int_t^{t+\Delta t} \sum_{i=1}^N \sum_j \delta(t' - t_i^j) dt'$

Table 2: Summary of common ways to quantify neural activity. Each measure captures a different property of spiking: rate, variability, temporal modulation, timing regularity, or population behavior.

12 A neural population perspective: from single neurons to rate models

Up to now, we have focused on how a single neuron responds to inputs and generates spikes. But **real computations in the brain are carried out by large populations of neurons acting together**. Instead of tracking every individual spike from every neuron, it is often more useful to describe the **collective activity** of the population. If the neurons in a group are similar (same properties and similar inputs), we can summarise their activity with a single quantity, the **mean firing rate** of the population:

$$A_0 = \langle \nu_i \rangle_i$$

where ν_i is the firing rate of neuron i .

In real networks this population activity is not constant: it varies over time because of noise, fluctuations in input, and the interactions between neurons. Rate models provide a convenient way to describe how such population activity evolves without needing to follow individual spikes.

12.1 The mean firing rate

To introduce population models, it helps to revisit the simple membrane equation of a leaky neuron:

$$\tau \frac{du}{dt} = -u + RI_{\text{ext}}$$

where $u(t)$ is the membrane potential and I_{ext} the external current. The steady firing rate of this neuron is given by a **gain function**:

$$\nu = g(I_0)$$

which expresses how input current is transformed into firing rate. **Instead of tracking the full voltage $u(t)$, we can directly describe the neuron by its firing rate:**

$$\nu(t) = g(I_{\text{ext}}(t))$$

A model neuron defined by its instantaneous firing rate is called a **rate neuron**. Rate models ignore the fine details of the membrane potential and **focus on how inputs control average firing rate**, an approach that becomes extremely useful when studying large populations.

12.2 The gain function

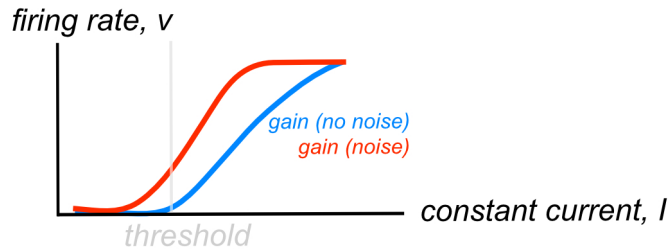


Figure 13: Gain functions of a rate neuron with and without noise. Noise makes the input–output curve smoother and more sigmoidal.

The **gain function** $g(I)$ describes how a neuron converts an input current I into a firing rate ν . For most neurons, **this relationship is nonlinear**: the neuron responds very little to weak inputs, increases its firing once a threshold is crossed, and eventually saturates at high inputs. A simple example is a threshold–linear function:

$$g(I_0) = \begin{cases} 0 & I_0 < I_{\text{th}} \\ \alpha (I_0 - I_{\text{th}}) & I_0 \geq I_{\text{th}} \end{cases}$$

In real neurons, however, **noise is always present**. Membrane potential fluctuates because of random synaptic input, channel noise, and other sources. If we model this noise explicitly:

$$\tau \frac{du}{dt} = -u + RI_{\text{ext}} + \eta(t)$$

the voltage jitters around its mean. **These fluctuations matter:** even when the average input is slightly below threshold, the voltage may occasionally fluctuate above threshold (η) and produce a spike. As a consequence, the effective **gain function becomes smoother**: instead of a sharp step at I_{th} , the firing rate increases gradually.

A convenient way to describe this smooth input–output relation is with a **sigmoid** (Figure 14):

$$\nu = \frac{\nu_{\text{max}}}{1 + e^{-\beta(I_0 - I_{\text{th}})}}$$

This sigmoidal form captures three key features of real gain curves:

- A **soft threshold** (firing begins gradually, not abruptly).
- A **steep region** where firing increases rapidly.
- **Saturation** at high inputs.

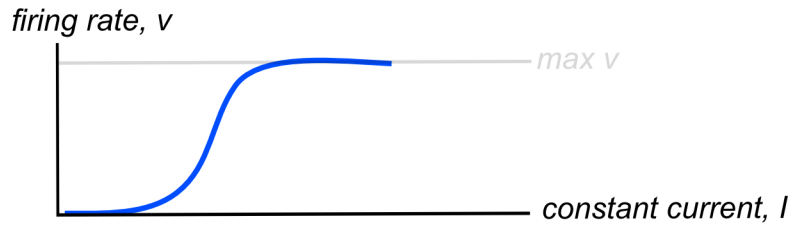


Figure 14: A sigmoidal approximation to the neuronal gain function.

12.3 The rate equation

So far we have described the steady-state (the firing rate it would reach if the input were held constant for a long time) firing rate of a neuron through the gain function $\nu = g(I)$. However, when the input changes over time, the neuron cannot instantly jump to this new value. **Real neurons need time to integrate the input, and their firing rate adjusts gradually.**

A simple way to model this behavior is to assume that the firing rate $\nu(t)$ relaxes toward its instantaneous steady-state value $g(I(t))$ with a characteristic time constant τ . This leads to the **rate equation**:

$$\tau \frac{d\nu}{dt} = -\nu + g(I(t)) \quad (16)$$

The form of this equation is intuitive:

- If $\nu(t)$ is below the desired value $g(I(t))$, then $d\nu/dt > 0$ and the rate increases.
- If $\nu(t)$ is above it, then $d\nu/dt < 0$ and the rate decreases.
- The time constant τ controls how quickly the neuron responds to changes.

Thus, the rate equation says that the neuron’s activity smoothly approaches the firing rate predicted by the gain function (introduced in Figure 13), rather than jumping to it instantly.

13 Recurrent rate networks

So far we have considered a neuron receiving only external input. We now explore what happens when a neuron (or a population) also receives **input from itself through recurrent connections**. This simple modification already produces rich and important dynamical behaviors.

If the population excites itself with synaptic strength w (Figure 15), the total input becomes:

$$I(t) = w \nu(t) + I_{\text{ext}}(t)$$

Substituting this expression into the rate equation gives the **recurrent rate model**:

$$\tau \frac{d\nu}{dt} = -\nu + g(w\nu + I_{\text{ext}}) \quad (17)$$

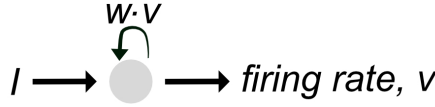


Figure 15: A rate neuron receiving recurrent input proportional to its own activity. Recurrent excitation feeds the neuron's past activity back onto itself.

This simple equation already reveals important network properties:

- **Recurrent excitation amplifies small inputs.** Even a weak external drive can be boosted by positive feedback.
- **Strong self-excitation can create multiple stable activity levels.** Depending on w , the network may settle into a low-activity or high-activity state.
- **The nonlinear shape of the gain function** (Figures 13–14) interacts with the feedback term $w\nu$ to shape the network's dynamics.

13.1 Bistability

To understand the long-term behavior of a recurrent rate network, we look for **fixed points** of the activity. A fixed point is a firing rate ν that remains constant over time, meaning:

$$\nu = g(w\nu + I)$$

A convenient way to find fixed points is to use a graphical analysis. We plot the line $y_1 = \nu$ and the curve $y_2 = g(w\nu + I)$, and look for intersections (Figure 16, left).

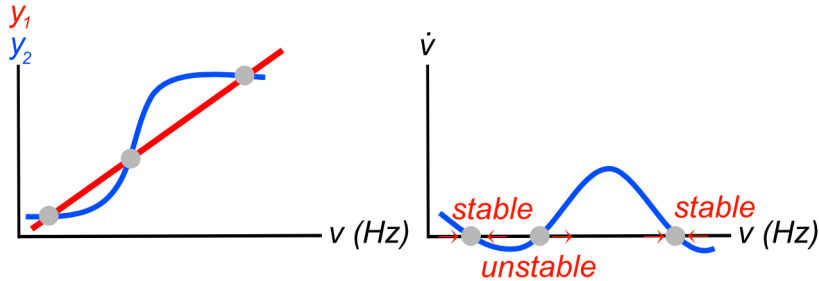


Figure 16: Graphical analysis of fixed points in a recurrent rate model. **Left:** Fixed points are found where the line $y_1 = \nu$ intersects the curve $y_2 = g(w\nu + I)$. Depending on w , there may be one or three intersections. **Right:** Plotting $\dot{\nu}$ reveals their stability. The low and high intersections are stable (attractors), while the middle one is unstable. Together, they form a bistable system with two persistent activity states.

With a sigmoidal gain function, the two curves can intersect either once or three times:

- **One intersection:** When the recurrent weight w is very small or very large, the gain curve is either too shallow or too steep, so it crosses $y_1 = \nu$ only once. The network then has only one stable activity level.
- **Three intersections:** For intermediate values of w , the gain curve becomes steep enough to intersect the line three times. The outer intersections correspond to **stable** fixed points, while the middle one is **unstable**. This creates two possible long-term activity levels.

When the system has two stable fixed points (one at low firing rate and one at high firing rate), the network is said to be **bistable**.

Why bistability behaves like memory. Each stable **fixed point acts as an attractor**: once the activity settles near it, the network remains there. A brief input can push the system from the low-activity attractor to the high-activity one. Even after the input disappears, the activity stays high, the network has effectively “remembered” that the input occurred. To return to the low state, a separate input is needed to cross the unstable middle fixed point.

Because a **bistable network can hold one of two persistent activity levels**, it behaves like an **ON/OFF memory switch**. This is why recurrent excitation and bistability are often used in models of working memory and persistent activity.

14 Interacting rate networks

We now explore how activity behaves when **two populations influence each other**. In the circuit diagrams that follow, we use the standard conventions from neural network models: arrows with a **pointed tip** represent **excitatory connections**, and arrows ending in a **flat bar** represent **inhibitory connections**. These symbols simply tell us whether one population tends to increase or decrease the activity of another.

Additionally, in rate models we write the synaptic weight from population X onto population Y as w_{YX} . The **subscript indicates the direction of the connection**: the first index is the target and the second index is the source. For example:

w_{IE} means “connection from E to I ”

This convention helps us immediately see which population is driving which, independent of whether the connection is excitatory (arrow head) or inhibitory (flat bar).

14.1 Excitatory–excitatory interactions: cooperative bistability

Consider two excitatory populations that excite each other (top panel of Figure 17). Their dynamics are described by:

$$\begin{cases} \dot{v}_1 = -\nu_1 + g(w\nu_2 + I) \\ \dot{v}_2 = -\nu_2 + g(w\nu_1 + I) \end{cases} \quad (18)$$

We can analyze the system using **nullclines** (bottom panel of Figure 17). Because the gain function is sigmoidal, both nullclines are S-shaped.

The intersections of the nullclines define the fixed points of the system. This network typically has:

- **Low–low stable state.** Both populations settle at low firing rates.
- **High–high stable state.** Both populations remain active at high rates.
- **Unstable middle fixed point.** Separates the two stable states.

Thus, the network is **bistable**: **both populations are either quiet together or active together**. This cooperative behavior appears in models of persistent activity and working memory.

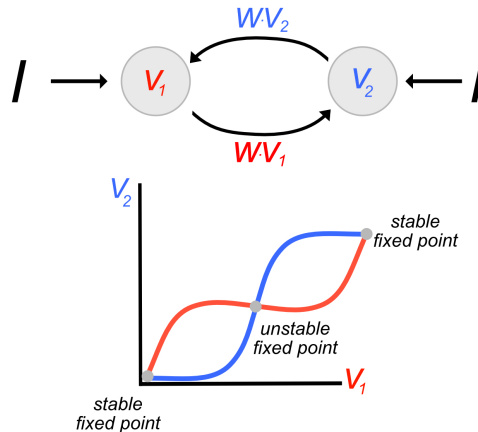


Figure 17: Two mutually excitatory rate populations (top), and their nullclines (bottom). Fixed points occur where the two curves intersect. Arrows indicate the direction of motion of the system.

14.2 Inhibitory–inhibitory interactions: competitive dynamics

Now consider two populations that inhibit each other (top panel of Figure 18). Their dynamics are:

$$\begin{cases} \dot{\nu}_1 = -\nu_1 + g(-w\nu_2 + I) \\ \dot{\nu}_2 = -\nu_2 + g(-w\nu_1 + I) \end{cases} \quad (19)$$

Here the coupling terms have negative sign, so each population suppresses the other. This flips the shape of the nullclines: instead of being S-shaped and rising, they slope downward (bottom panel of Figure 18).

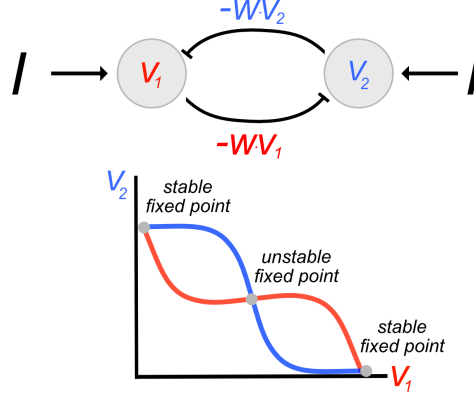


Figure 18: Two mutually inhibitory rate populations (top), and their nullclines (bottom). Mutual inhibition produces two stable states in which one population dominates and the other is suppressed.

The nullclines intersect at three fixed points. The resulting behavior is:

- **High-low stable state.** Population 1 suppresses population 2.
- **Low-high stable state.** Population 2 suppresses population 1.
- **Unstable middle fixed point.** Separates the two stable outcomes.

This is a **competitive network**: one population “wins” and the other is suppressed. Such winner-take-all dynamics are important in models of decision-making, where two neural populations compete and only one remains active.

14.3 Excitatory–inhibitory interactions: balanced networks

We now examine a more realistic situation in which one excitatory population E interacts with an inhibitory population I . Recordings in cortex show that the firing rates of these two populations are often strongly correlated: when the excitatory population becomes more active, the inhibitory population usually increases its firing as well [9]. But, why does this happen? **Although inhibition opposes excitation, inhibitory neurons depend on excitatory input to fire.** When E becomes more active, it drives I more strongly, so their firing rates tend to rise and fall together.

The dynamics of two interacting populations can be written as:

$$\begin{cases} \dot{\nu}_E = -\nu_E + g(w_{EE}\nu_E - w_{EI}\nu_I + I^E) \\ \dot{\nu}_I = -\nu_I + g(w_{IE}\nu_E - w_{II}\nu_I + I^I) \end{cases} \quad (20)$$

Here w_{EE} and w_{IE} are excitatory connections, while w_{EI} and w_{II} are inhibitory. External inputs I^E and I^I shift the nullclines for E and I .

The two nullclines $\dot{\nu}_E = 0$ and $\dot{\nu}_I = 0$ intersect at a single fixed point. Because E drives I and I feeds back onto E , this fixed point lies on a part of the curves where both activities tend to increase together.

How changes in input move the fixed point If we slowly change the external input I^E or I^I , the corresponding nullcline shifts upward or downward (thin lines in Figure 19):

- **Increasing I^E shifts the E nullcline upward.** The fixed point moves up and to the right, increasing both ν_E and ν_I .
- **Increasing I^I shifts the I nullcline upward.** The fixed point moves down and to the left, decreasing both ν_E and ν_I .

In both cases, the fixed point moves in a direction where E and I change together.

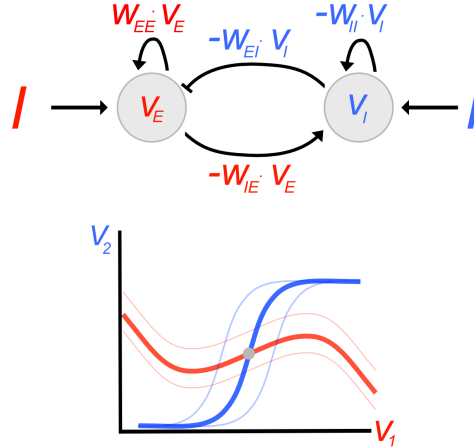


Figure 19: Nullclines of an excitatory–inhibitory rate model. Their intersection is the fixed point of the system. Moving the nullclines by changing external input shifts the fixed point and therefore changes both ν_E and ν_I .

Why E and I are correlated Because inhibitory neurons receive strong excitatory input, any increase in ν_E drives an increase in ν_I . Likewise, when excitation decreases, inhibition also receives less drive and decreases. Thus the **E–I network naturally produces the positive correlation** between excitatory and inhibitory firing rates that is **widely observed in cortical recordings**.

Center–surround modulation (Ozeki et al., 2009)

A well-known example of E – I co-modulation comes from experiments in visual cortex [10]. If the **animal is shown a small visual stimulus, the excitatory population E becomes active**, and this activity drives the inhibitory population I as well. Both groups of neurons fire strongly. If the **stimulus is made larger**, something different happens: the **responses of both E and I become smaller** (Figure 20).

Then, why does enlarging the stimulus reduce activity? The larger stimulus activates an additional “surround” pathway that sends extra inhibition (or less excitation) to the excitatory population E . In our E – I rate model, this corresponds to **reducing the external input I^E** .

When I^E decreases, the E population fires less. Because I relies on excitation to drive its own activity, the inhibitory population also fires less. Thus both firing rates drop together:

$$\text{larger stimulus} \Rightarrow \text{less input to } E \Rightarrow \text{smaller } \nu_E \Rightarrow \text{smaller } \nu_I$$

The key idea is that **inhibitory neurons reduce their firing not because they become “weaker,” but because they receive less excitatory input**. This simple interaction explains why both excitatory and inhibitory neurons decrease their activity when a larger visual stimulus engages the surround.

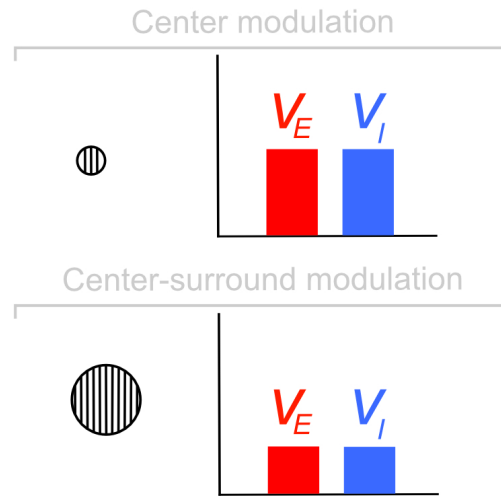


Figure 20: Center versus center-surround modulation. **Top:** A small central stimulus increases activity in both excitatory (v_E) and inhibitory (v_I) populations. **Bottom:** A large stimulus that activates both the center and the surround produces weaker responses, reflecting additional surround inhibition.

15 Decision making

15.1 The random-dot motion task

To illustrate how rate models can describe perceptual decisions, we use the **random-dot motion task**. A perceptual decision is a choice made on the basis of noisy or ambiguous sensory information (for example, deciding whether a group of dots is drifting left or right when the visual signal is weak).

In this task, the subject views a cloud of dots on a screen. Most dots move randomly, while a subset moves coherently either to the left or to the right. The fraction of dots that move together defines the **motion coherence**:

- **Low coherence:** only a small fraction of dots move in the same direction, making the overall motion difficult to perceive.
- **High coherence:** a large fraction of dots move in the same direction, making the motion direction clear and easy to identify.

The subject (typically a monkey) must decide whether the overall motion is to the left or to the right and report the decision with an eye movement. Behaviorally, performance depends strongly on motion coherence: at low coherence, choices are close to the chance level (i.e. the accuracy expected from random guessing), whereas at high coherence accuracy approaches 100%.

Recordings from visual motion-sensitive areas show that many **neurons prefer motion in particular directions and respond more strongly when the dots move in their preferred direction**. However, these sensory responses do not by themselves determine the decision. The final choice is believed to be formed downstream, in circuits that accumulate, compare, and amplify the small differences in sensory evidence provided by these neurons.

15.2 A three-population competitive E–I network

We can describe this behavior with a small network made of:

- **Two excitatory populations:**

$$\nu_E^L \text{ (prefers left),} \quad \nu_E^R \text{ (prefers right)}$$

- **One shared inhibitory population** ν_I that creates competition between them.

Each excitatory population excites itself and activates the inhibitory population (Figure 21). The inhibitory population feeds back onto both excitatory groups and suppresses them. This structure means that if one excitatory population becomes slightly stronger, it quickly suppresses the other.

Even if both sides receive equal input ($I^L = I^R$), the network does not settle halfway. Small random fluctuations are amplified: one side becomes a little stronger, recruits more inhibition, and “wins.”

The behavior of the network can be summarized as:

- **If motion evidence supports “left”:** $I^L > I^R \rightarrow \nu_E^L$ grows $\rightarrow$ suppresses $\nu_E^R \rightarrow$ choose left.
- **If motion evidence supports “right”:** $I^R > I^L \rightarrow \nu_E^R$ grows $\rightarrow$ suppresses $\nu_E^L \rightarrow$ choose right.
- **If evidence is weak or ambiguous:** small fluctuations decide which population wins, just like in behaviour at low coherence.

Thus, the network has two stable outcomes:

$$\nu_E^L \gg \nu_E^R \quad (\text{choose left}), \quad \nu_E^R \gg \nu_E^L \quad (\text{choose right})$$

This simple competitive circuit captures key features of perceptual decision making: smooth dependence on stimulus strength, probabilistic choices at low coherence, and clear “winner-take-all” outcomes.

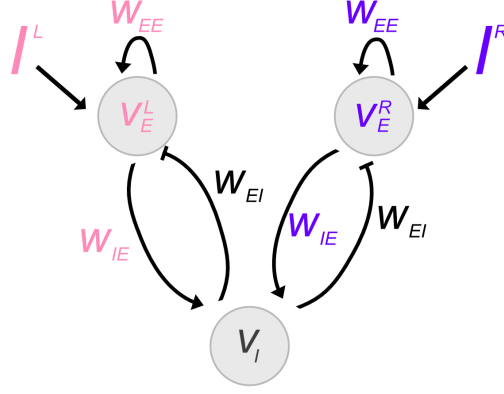


Figure 21: Competitive E - I circuit used to model decisions in the random-dot motion task. Two excitatory populations represent the “left” and “right” choices and receive inputs I^L and I^R . Both populations excite a shared inhibitory population, which feeds back inhibition to each excitatory group. This common inhibition creates competition: small differences in input are amplified so that only one excitatory population “wins” and determines the decision.

15.3 Equations of the decision circuit and model reduction

We now write the dynamical equations corresponding to the circuit in Figure 21:

$$\begin{cases} \tau_E \dot{\nu}_E^L = -\nu_E^L + w_{EE} g_E(\nu_E^L) + w_{EI} g_I(\nu_I) + RI^L \\ \tau_E \dot{\nu}_E^R = -\nu_E^R + w_{EE} g_E(\nu_E^R) + w_{EI} g_I(\nu_I) + RI^R \\ \tau_I \dot{\nu}_I = -\nu_I + w_{IE} g_E(\nu_E^L) + w_{IE} g_E(\nu_E^R) \end{cases} \quad (21)$$

Here:

- τ_E and τ_I are the time constants of excitatory and inhibitory populations.
- $g_E(\cdot)$ and $g_I(\cdot)$ are their gain (input-output) functions.
- $w_{EE} > 0$ is recurrent excitation within each excitatory pool.
- $w_{IE} > 0$ is excitation from E onto I .
- $w_{EI} < 0$ is inhibition from I back onto E .
- RI^L and RI^R are external inputs encoding left/right motion evidence.

For simplicity, **we reduce the three-variable system to a two-variable model that includes only the excitatory populations.** This is done as follows.

Step 1: Separation of time scales

We assume that **inhibition reacts much faster than excitation:**

$$\tau_I \ll \tau_E$$

This implies that, on the slower excitatory time scale, the inhibitory population remains close to its steady state. We therefore set:

$$\dot{\nu}_I \approx 0$$

Solving the inhibitory steady-state condition gives:

$$\nu_I \approx w_{IE} [g_E(\nu_E^L) + g_E(\nu_E^R)]$$

That is, the **inhibitory population activity is proportional to the sum of the excitatory activities**, weighted by w_{IE} .

Step 2: Linear approximation of the inhibitory gain

Next, we approximate the **inhibitory gain function by a straight line** over the relevant operating range:

$$g_I(\nu_I) \approx \gamma \nu_I$$

where $\gamma > 0$ controls how strongly inhibition grows with its input. Combining this approximation with the result from Step 1 yields:

$$g_I(\nu_I) \approx \gamma w_{IE} [g_E(\nu_E^L) + g_E(\nu_E^R)]$$

Step 3: From inhibitory feedback to an effective coupling α

The inhibitory feedback received by each excitatory population appears in the excitatory equations as the term $w_{EI}g_I(\nu_I)$. Using the approximation above, this becomes:

$$w_{EI}g_I(\nu_I) \approx w_{EI} \gamma w_{IE} [g_E(\nu_E^L) + g_E(\nu_E^R)]$$

We now examine the signs of the parameters:

$$w_{EI} < 0, \quad w_{IE} > 0, \quad \gamma > 0$$

Their product is therefore negative. To simplify notation and make the inhibitory effect explicit, we define a positive constant:

$$\alpha = -\gamma w_{EI} w_{IE} > 0$$

With this definition, the inhibitory feedback can be written as:

$$w_{EI}g_I(\nu_I) = -\alpha g_E(\nu_E^L) - \alpha g_E(\nu_E^R)$$

This form highlights two effects:

- **Each excitatory population receives inhibition proportional to its own activity** (self-inhibition via the shared inhibitory pool).
- **Each population also receives inhibition proportional to the activity of the other population** (mutual inhibition), with the same strength α .

Step 4: Reduced two-dimensional system

We now substitute this inhibitory feedback term into the excitatory equations. For the left population, the terms involving $g_E(\nu_E^L)$ combine as:

$$w_{EE} g_E(\nu_E^L) - \alpha g_E(\nu_E^L) = (w_{EE} - \alpha) g_E(\nu_E^L)$$

and the **cross term** $-\alpha g_E(\nu_E^R)$ **represents inhibition from the right pool**. A similar structure holds for the right population. This yields a closed system for the two excitatory populations:

$$\boxed{\begin{aligned} \tau_E \dot{\nu}_E^L &= -\nu_E^L + (w_{EE} - \alpha) g_E(\nu_E^L) - \alpha g_E(\nu_E^R) + R I^L \\ \tau_E \dot{\nu}_E^R &= -\nu_E^R + (w_{EE} - \alpha) g_E(\nu_E^R) - \alpha g_E(\nu_E^L) + R I^R \end{aligned}} \quad (22)$$

In this reduced description:

- $(w_{EE} - \alpha) g_E(\nu_E^{L,R})$ is the **effective self-excitation** of each population (recurrent excitation reduced by the shared inhibition).
- $-\alpha g_E(\nu_E^{R,L})$ is the **effective mutual inhibition** between the two excitatory populations.

Thus, on a slower time scale, the original three-population E - I circuit behaves like **two competing excitatory populations that inhibit each other through an effective coupling α** .

15.4 Dynamics of decision making

We can understand how this decision circuit behaves by analyzing the nullclines of the two excitatory populations, ν_E^L and ν_E^R . The fixed points of the system correspond to the intersections of these nullclines. Depending on the external inputs I^L and I^R , the system behaves in three qualitatively different ways.

1. No inputs ($I^L = I^R = 0$). With no external drive, both excitatory populations only receive inhibition (Figure 22, top). The nullclines intersect once, close to:

$$\nu_E^L = 0, \quad \nu_E^R = 0$$

This point is **stable**: both populations remain silent because neither receives enough excitation to become active. In other words, the network does not “choose” either side, it stays in a neutral, **no-decision state** with both activities near zero.

2. Strong input to the left population only ($I^L > 0, I^R = 0$). Increasing the left input shifts the left nullcline upward (Figure 22, middle). The stable fixed point moves to a location where:

$$\nu_E^L \text{ is high, } \quad \nu_E^R \text{ is low}$$

In other words, the left population becomes active and suppresses the right population through the shared inhibition. The network therefore settles into the **“choose left” state**.

3. Strong inputs to both populations ($I^L > 0, I^R > 0$). When both sides receive strong evidence, both nullclines shift (Figure 22, bottom). The system now has:

- A stable state with $\nu_E^L \gg \nu_E^R$ (choose left).
- A stable state with $\nu_E^R \gg \nu_E^L$ (choose right).
- An unstable middle fixed point separating them.

Because the inputs are similar, the network is almost balanced. **Small fluctuations** (coming from noise or tiny differences in initial activity) **determine which population becomes dominant**. This corresponds to behavior at **low coherence**: the motion signal is very weak, so the network has almost no clear information about the direction, and the decision changes from trial to trial.

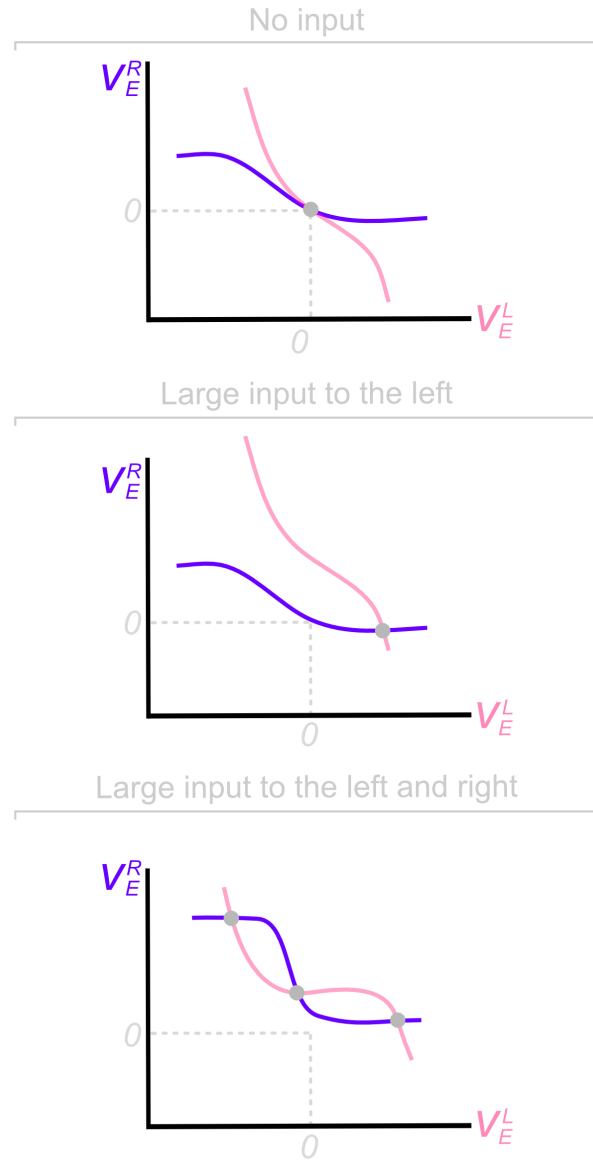


Figure 22: Nullcline diagrams illustrating the three possible behaviors of the decision circuit. **Top:** With no external input, the nullclines intersect once near the origin, and both populations remain inactive (no decision). **Middle:** A strong input to the left shifts the left nullcline, creating a stable fixed point with high ν_E^L and low ν_E^R (decision to the left). **Bottom:** Strong inputs to both sides produce two stable fixed points separated by an unstable one. Small fluctuations determine which population wins, as in low-coherence trials.

16 Continuous Population Models

16.1 The ring network

Up to now, we have worked with networks containing only a few distinct populations. We now consider a different type of organisation: a **continuous population**. Here, neurons are arranged along a smooth axis (such as orientation preference), so that **nearby neurons respond to similar stimuli while distant neurons respond to very different ones**.

A classic example is primary visual cortex (V1), where neurons are tuned to all possible orientations of a bar of light. Because orientation varies smoothly from $-\pi$ to π , we label each neuron by a continuous variable θ representing its preferred orientation.

Neurons with **similar** preferred orientations tend to **co-activate**, whereas neurons with **very different** preferred orientations tend to **suppress** each other through inhibitory circuits. Because orientation wraps around (e.g. 0° and 180° represent the same axis), the population naturally forms a circle. Models based on this circular arrangement are known as **ring networks** [11].

16.2 Rate equation on the ring

Each preferred orientation θ has an associated firing rate $\nu(\theta, t)$ (Figure 23). The dynamics are:

$$\tau \frac{d\nu(\theta, t)}{dt} = -\nu(\theta, t) + g \left(\int_{-\pi}^{\pi} W(\theta - \theta') \nu(\theta', t) d\theta' + I(\theta) \right) \quad (23)$$

- τ is the time constant.
- g is the gain (activation) function.
- $W(\theta - \theta')$ is the influence of neurons tuned to θ' on neurons tuned to θ .
- The integral replaces a sum because the population is continuous.

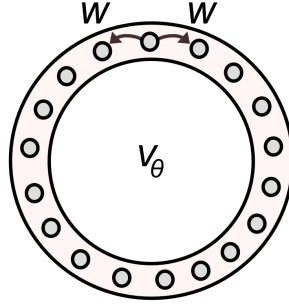


Figure 23: Schematic of the ring model. Neurons are arranged on a circle according to their preferred orientation θ . Neurons with nearby orientations interact more strongly, while far-apart orientations interact weakly or suppressively.

16.3 Cosine-shaped connectivity

A widely used assumption is that the strength of interaction between two neurons depends only on the **difference in their preferred orientations**. At a conceptual level, this can be written as:

$$W(\theta - \theta') = W_0 + W_1 \cos(\theta - \theta').$$

This cosine form captures the idea that neurons with similar preferences interact differently from neurons with dissimilar preferences. In the next subsection, we introduce the classical ring model and show how this expression is modified to account for the π -periodicity of orientation.

Why a cosine? Writing $W(\theta - \theta')$ as a cosine implicitly allows the same neuron to exert either a positive (excitatory) or negative (inhibitory) influence on another neuron. Biologically, this is a simplification as real neurons are either excitatory or inhibitory, not both. Here, however, $W(\theta - \theta')$ represents the **net effect** of all multi-synaptic pathways, not a single synapse.

How to interpret the cosine shape.

- **Nearby orientations excite each other.** If $\theta \approx \theta'$, then $\cos(\theta - \theta') > 0$, so $W(\theta - \theta')$ is positive (net excitation).
- **Very different orientations inhibit each other.** If $|\theta - \theta'|$ is large, then $\cos(\theta - \theta') < 0$, so $W(\theta - \theta')$ becomes negative (net inhibition).

Thus, a single mathematical function encodes an important cortical principle:

Nearby orientations cooperate; distant orientations compete.

This cosine form is mathematically convenient and biologically motivated, making it the standard choice in ring models.

16.4 The original ring model

In the classical formulation of the ring model [11], the stimulus input to a neuron tuned to orientation θ is:

$$I(\theta) = C[(1 - \epsilon) + \epsilon \cos(2(\theta - \theta_s))] \quad (24)$$

Here:

- C is the stimulus contrast.
- ϵ determines how sharply the input is tuned.
- θ_s is the stimulus orientation.

The factor of 2 appears because orientation is periodic every 180° : a bar at θ_s and a bar at $\theta_s + \pi$ produce the same input. Using $\cos(2(\cdot))$ ensures this symmetry.

The recurrent connectivity has a similarly structured form:

$$W(\theta - \theta') = -W_0 + W_1 \cos(2(\theta - \theta')) \quad (25)$$

- The term $-W_0$ represents broad, untuned inhibition.
- The term $W_1 \cos(2(\theta - \theta'))$ implements tuned, orientation-specific cooperation.

What does the cosine term do?

- If $\theta \approx \theta'$, the cosine is positive. **This yields tuned recurrent excitation between similarly tuned neurons.**
- If θ and θ' are far apart, the cosine becomes negative. **This yields a net suppressive effect**, typically mediated by inhibitory interneurons.

Thus, W_1 sets the amplitude of the tuned component: large W_1 means strong center-excitation and strong surround-inhibition.

With these components, the ring model reproduces two hallmark properties of V1:

- **Contrast invariance.** As contrast C increases, the input $I(\theta)$ becomes broader, but the network response remains sharply tuned. Recurrent excitation enhances the centre of the tuning curve, while global inhibition suppresses the flanks, the “iceberg effect” (Figure 24).
- **Sustained activity.** If recurrent excitation is sufficiently strong, a bump of activity can persist even after the stimulus disappears, a form of stable, self-sustained representation.

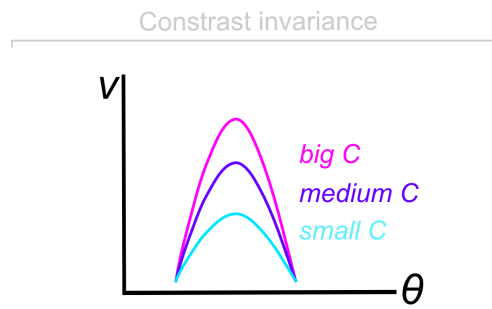


Figure 24: Contrast invariance. Increasing stimulus contrast (C) increases response amplitude but leaves the tuning width nearly unchanged.

Overall, the ring model shows how a simple combination of tuned excitation and broad inhibition along a circular orientation axis is enough to generate **sharp tuning**, **contrast invariance**, and even **sustained activity**.

17 Neural networks of spiking neurons

Rate models describe population activity through a continuous variable (the firing rate), which is useful for studying large-scale dynamics. However, real neurons communicate through discrete action potentials, and the timing of these spikes can strongly influence network behavior. In this section we introduce **spiking neuron networks**, which model dynamics at the level of individual spikes.

A remarkable feature of cortical activity is that spikes are **highly irregular**. Even when a neuron fires at a stable average rate, the exact timing of its spikes varies greatly. A standard measure of this variability is the coefficient of variation (CV) of the inter-spike intervals:

$$CV = \frac{\sigma_{\text{ISI}}}{\mu_{\text{ISI}}}$$

In cortex, one often finds:

$$CV \approx 1$$

A value near 1 indicates **Poisson-like behavior**: spike times are unpredictable, ISIs follow an exponential distribution, and only the average firing rate is stable.

But, where does this variability come from? As we will see, **large recurrent networks of excitatory and inhibitory neurons can generate irregular spiking intrinsically**, even in the absence of external noise. This is the essence of the **balanced state**.

17.1 Single-neuron dynamics

We begin with the LIF neuron. Its membrane potential $u(t)$ obeys:

$$\tau_m \frac{du}{dt} = -(u - u_{\text{rest}}) + RI^{\text{ext}} + RI^{\text{net}}$$

where:

- τ_m is the membrane time constant.
- u_{rest} is the resting potential.
- I^{ext} is external current.
- I^{net} is synaptic input from other neurons.

Each presynaptic spike contributes a stereotyped current pulse. The total synaptic input is:

$$I^{\text{net}}(t) = \sum_{k=1}^N w_k \int_0^\infty \epsilon(s) S_k(t-s) ds$$

Here:

- w_k is the synaptic weight of neuron k .
- $S_k(t) = \sum_j \delta(t - t_j^k)$ is its spike train.
- $\epsilon(s)$ is the synaptic kernel (response to a single spike).

A common choice is an **exponential kernel**:

$$\epsilon(s) = e^{-s/\tau_m} \Theta(s)$$

meaning a spike produces an instantaneous jump followed by exponential decay.

17.2 Large recurrent networks

We now consider a network with many excitatory (E) and inhibitory (I) neurons (Figure 25). To keep the model analytically tractable we use the following standard assumptions:

- **All neurons have similar biophysical properties.**
- **Connectivity is random but statistically uniform.** Each neuron receives roughly the same number of excitatory (K_E) and inhibitory (K_I) inputs.
- **Both K_E and K_I are large and scale with network size:**

$$K_E \sim K_I \sim K$$

The connection probability is:

$$\varepsilon = \frac{K_E}{N_E} = \frac{K_I}{N_I}$$

typically with $\varepsilon \ll 1$: each neuron samples only a small fraction of the population.

We denote average firing rates by:

$$r_x \quad (\text{external input}), \quad r_E \quad (\text{excitatory}), \quad r_I \quad (\text{inhibitory})$$

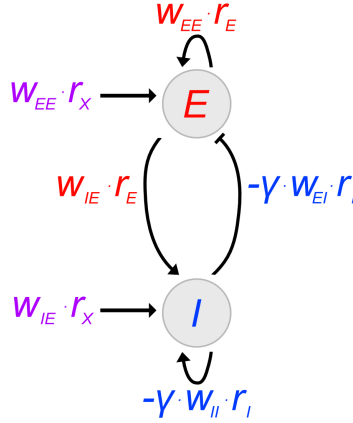


Figure 25: Structure of the E - I network used in the balanced-state model. The excitatory population E receives external drive, recurrent excitation, and inhibition from I . The inhibitory population I receives external drive, excitation from E , and recurrent inhibition. Arrow colours indicate the type of input: excitatory (red), inhibitory (blue), and external (purple).

17.3 Synaptic input: mean and fluctuations

Each neuron receives many small inputs. The total input can be decomposed into:

$$\text{mean input} \quad + \quad \text{fluctuations}$$

Both depend on the number of incoming synapses K .

Mean input. The mean synaptic current to an excitatory neuron is:

$$\boxed{\mu_E = K [w_{EE}(r_x + r_E) - \gamma w_{EI}r_I]} \quad (26)$$

Likewise, for an inhibitory neuron:

$$\mu_I = K [w_{IE}(r_x + r_E) - \gamma w_{II}r_I]$$

In both cases:

$$\text{mean input} = (\text{number of synapses}) \times (\text{total excitation} - \text{total inhibition})$$

where γ scales the relative strength of inhibitory input (e.g. due to stronger inhibitory synapses or higher inhibitory firing rates).

A note on the mathematical approximation. In writing the mean inputs μ_E and μ_I , we implicitly used an important simplification: each neuron receives a very large number of synaptic inputs ($K \gg 1$), and these inputs are statistically independent across presynaptic neurons. By the **central limit theorem**, the sum of many small, independent synaptic contributions behaves like a Gaussian random variable (any small independent inputs average out to noise with a well-defined mean and variance). This justifies treating the total synaptic input as a Gaussian process and splitting it into a **mean part** (given by μ_E and μ_I) and a **fluctuating part** whose variance we compute next.

Input fluctuations. Because a neuron receives many independent spike trains, the variability of its input scales as:

$$\sigma^2 \propto K$$

A more explicit expression for excitatory neurons is:

$$\sigma_E^2 = K (w_{EE}^2(r_x + r_E) + \gamma w_{EI}^2 r_I) \quad (27)$$

so:

$$\sigma_E \propto \sqrt{K}$$

This difference between how mean input ($\sim K$) and fluctuations ($\sim \sqrt{K}$) scale is crucial.

17.4 Weak vs. strong coupling

How synaptic weights scale with K determines network behavior.

Case I: Weak coupling ($w \sim 1/K$).

$$\mu \sim 1, \quad \sigma \sim \frac{1}{\sqrt{K}}$$

Fluctuations vanish as the network grows. **Neurons fire almost deterministically (not what we see in the cortex).**

Case II: Strong coupling ($w \sim 1/\sqrt{K}$).

$$\mu \sim \sqrt{K}, \quad \sigma \sim \text{constant}$$

Here fluctuations remain substantial, but the mean input diverges unless excitation and inhibition nearly cancel. This leads us naturally to the balanced state.

17.5 The balanced state

A **balanced state** arises when **strong excitation and strong inhibition cancel on average**:

$$\begin{aligned} w_{EE}(r_x + r_E) - \gamma w_{EI} r_I &= 0 \\ w_{IE}(r_x + r_E) - \gamma w_{II} r_I &= 0 \end{aligned}$$

When these balance equations hold:

- The mean input remains small (because excitation and inhibition cancel).
- Fluctuations remain finite (because weights scale as $1/\sqrt{K}$).
- Neurons fire irregularly, driven by fluctuations rather than mean input.

This produces:

$$CV \approx 1$$

matching the irregular, **Poisson-like spiking** widely observed in cortical populations.

The balanced state therefore provides a simple, powerful explanation of **how large recurrent networks generate irregular activity without any external noise**.

18 Unsupervised learning

18.1 Introduction to learning and memory

The basis of learning in neural systems is **synaptic plasticity**: the ability of **synapses to change their strength** (weight w) in response to neural activity. Changes in synaptic strength allow circuits to store information, form associations, and extract statistical structure from sensory input.

A classic example is **associative memory**: when activity patterns that repeatedly co-occur become linked through strengthened synapses. Over time, activating part of the pattern can reactivate the entire stored “concept”. Learning algorithms are often grouped into three broad families:

- **Unsupervised learning.** Learning the statistics and structure of the input without labels.
- **Reinforcement learning.** Learning to choose actions based on reward or punishment.
- **Supervised learning.** Using a teacher signal or target output to guide synaptic changes.

In this chapter we focus on **unsupervised learning** and Hebbian plasticity, which provide a simple yet powerful framework for understanding how sensory systems develop structured representations.

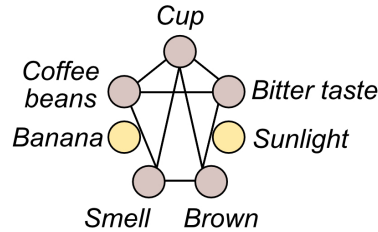


Figure 26: Associative memory network for a “coffee” concept. Nodes represent features or items, and edges indicate Hebbian associations formed through repeated co-activation. The central cluster (cup, coffee beans, smell, brown, bitter taste) consists of features that reliably co-occur and therefore become strongly interconnected. The yellow nodes (banana, sunlight) show weak or no co-activation with the central features and remain poorly connected, illustrating how associative memories selectively link statistically related inputs.

18.2 Hebbian learning: “cells that fire together, wire together”

Hebbian learning originates from a proposal by Donald Hebb [12]:

“When an axon of cell A is near enough to excite cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency, as one of the cells firing B, is increased”

Modern summaries often express this idea as:

“Cells that fire together, wire together”

18.3 Rate-based formalism

Consider a **synapse from neuron i to neuron j with weight w_i** . Let x_i denote the presynaptic firing rate and y the postsynaptic firing rate. A general activity-dependent learning rule is:

$$\frac{dw_i}{dt} = F(w_i, x_i, y)$$

where F describes how the synapse changes as a function of pre- and postsynaptic activity. To understand the possible structure of the learning rule F , we expand it in a **Taylor series** around the baseline activity $x_i = y = 0$. This expansion **decomposes synaptic plasticity into contributions driven by different combinations of pre- and post-synaptic activity**:

- A constant term (activity-independent weight drift).
- A term proportional to x_i (presynaptic dependence).

- A term proportional to y (postsynaptic dependence).
- A mixed term proportional to $x_i y$ (co-activation or correlation).
- Higher-order nonlinear terms.

Formally, the Taylor expansion of $F(w_i, x_i, y)$ around $x_i = y = 0$ yields:

$$\frac{dw_i}{dt} = c_0(w_i) + c_i^{\text{pre}}(w_i) x_i + c_i^{\text{post}}(w_i) y + c_i^{xx}(w_i) x_i^2 + c_i^{yy}(w_i) y^2 + \boxed{c_i^{\text{corr}}(w_i) x_i y} + O(x_i^3, y^3)$$

Here, the coefficients $c_i^{(\cdot)}(w_i)$ are the Taylor coefficients of the learning rule F with respect to pre- and post-synaptic activity. Each coefficient quantifies how synaptic changes at synapse i depend on a specific combination of presynaptic activity x_i and postsynaptic activity y , and may depend on the current synaptic weight w_i .

The mixed term $x_i y$ captures how synapses change when **both the presynaptic and postsynaptic neurons are active**. The coefficient $c_i^{\text{corr}}(w_i)$ sets the strength and sign of correlation-based plasticity at synapse i . Retaining only this term yields the **simplest Hebbian learning rule**:

$$\boxed{\frac{dw_i}{dt} = c_i^{\text{corr}} x_i y} \quad (28)$$

18.4 Hebbian learning extracts input correlations

We now examine what structure the pure Hebbian rule extracts from its inputs. Assume that the **postsynaptic firing rate is a linear combination of presynaptic activities**:

$$y = \sum_i w_i x_i$$

This is a standard assumption in rate models and lets us see how Hebbian plasticity interacts with the statistics of the inputs. Starting from the simplest Hebbian term (synapses strengthen in proportion to the co-activation of pre- and post-synaptic neurons):

$$\frac{dw_i}{dt} \propto x_i y$$

we substitute $y = \sum_j w_j x_j$:

$$\frac{dw_i}{dt} \propto x_i \left(\sum_j w_j x_j \right) = \sum_j w_j x_i x_j$$

If weights evolve slowly, we can replace instantaneous products by their time averages:

$$\tau_w \frac{dw_i}{dt} = c^{\text{corr}} \sum_j \langle x_i x_j \rangle w_j$$

The quantity:

$$Q_{ij} = \langle x_i x_j \rangle$$

is the **input correlation matrix**. Each entry tells us how often input i and input j are active together. In vector notation, the learning rule becomes:

$$\tau_w \frac{d\vec{w}}{dt} = c^{\text{corr}} Q \vec{w}$$

Interpretation: why correlations matter. This equation tells us that Hebbian learning changes the weight vector $\vec{w}$ by repeatedly multiplying it by the correlation matrix Q . To understand what this does, we use the concepts of **eigenvectors** and **eigenvalues**.

An eigenvector $\vec{v}_k$ of Q is a special direction that is only stretched, not rotated, by Q :

$$Q \vec{v}_k = \lambda_k \vec{v}_k$$

Here:

- $\vec{v}_k$ is an eigenvector (a special pattern of synaptic weights).
- λ_k is the corresponding eigenvalue (how strongly this pattern is amplified).

Any weight vector can be written as a combination of eigenvectors:

$$\vec{w} = \sum_k a_k \vec{v}_k$$

Substituting this into the learning rule:

$$\tau_w \frac{d\vec{w}}{dt} = c^{\text{corr}} Q \vec{w} = c^{\text{corr}} Q \left(\sum_k a_k \vec{v}_k \right) = c^{\text{corr}} \sum_k a_k Q \vec{v}_k = c^{\text{corr}} \sum_k a_k \lambda_k \vec{v}_k$$

Comparing coefficients along each eigenvector shows how the amplitudes a_k evolve:

$$\tau_w \frac{da_k}{dt} = c^{\text{corr}} \lambda_k a_k$$

Thus:

- Each eigenvector $\vec{v}_k$ represents an independent “correlation pattern” in the inputs.
- Its coefficient a_k grows or shrinks at a rate set by the eigenvalue λ_k .
- The eigenvector with the largest eigenvalue grows the fastest and eventually dominates $\vec{w}$.

Then, **Hebbian learning makes the neuron most sensitive to the strongest correlation pattern in its inputs**. This is why the direction of $\vec{w}$ ends up aligned with the **dominant eigenvector of Q** . It is also the mathematical basis for later results such as:

- Oja’s rule performing principal component analysis (PCA).
- BCM learning leading to selective receptive fields for specific input patterns.

18.5 Why pure Hebbian learning is unstable

A rule of the form:

$$\frac{dw_i}{dt} \propto x_i y$$

contains **no mechanism that decreases or bounds the weight**. This creates a runaway feedback loop.

Example (runaway growth). To see why pure Hebbian learning is unstable, consider the simplest possible case. Assume that the presynaptic neuron fires at a constant rate:

$$x_i = 1$$

and that the postsynaptic activity increases proportionally to the synaptic weight:

$$y = w_i$$

This linear relation is a toy model: a stronger synapse produces a larger postsynaptic response.

With the pure Hebbian rule:

$$\frac{dw_i}{dt} = \eta x_i y$$

where $\eta > 0$ is a learning rate, we substitute $x_i = 1$ and $y = w_i$ to obtain:

$$\frac{dw_i}{dt} = \eta w_i$$

The solution is:

$$w_i(t) = w_i(0) e^{\eta t}$$

This demonstrates the core instability of pure Hebbian learning: a slightly larger weight produces slightly larger postsynaptic activity, which increases the Hebbian product $x_i y$, which in turn makes the weight grow even more. The **result is an exponential, runaway feedback loop**.

Biologically, synapses cannot increase without bound, so the **pure Hebbian rule must be modified with stabilizing mechanisms** such as normalization, mean subtraction, or activity thresholds (Table 3).

18.6 The covariance rule: Hebbian learning with baseline subtraction

Pure Hebbian learning strengthens synapses whenever activity is non-zero. However, **neurons typically have a baseline firing rate**. If plasticity acted directly on raw firing rates, **weights would increase even during spontaneous activity**.

To prevent this, we subtract the mean (baseline) activity and learn only from **fluctuations around baseline**. We define:

$$\tilde{x}_i = x_i - \theta_x, \quad \tilde{y} = y - \theta_y$$

where:

$$\theta_x = \langle x_i \rangle, \quad \theta_y = \langle y \rangle$$

represent the **temporal averages** of the pre- and postsynaptic activity. The tilde therefore denotes **deviation from baseline activity**. Using these centered variables, the covariance learning rule becomes:

$$\tau_w \frac{d\vec{w}}{dt} = \tilde{y} \tilde{\vec{x}} \quad (29)$$

where $\tilde{\vec{x}} = (\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_N)$ is the vector of baseline-subtracted presynaptic activities.

Interpretation.

- **Synapses strengthen** when both presynaptic and postsynaptic activity are **above their baseline levels**.
- **Synapses weaken** when either activity is **below baseline**.
- Baseline firing does not change the weights because $\tilde{x}_i = \tilde{y} = 0$ at baseline.

Thus, the covariance rule captures **statistical co-fluctuations in activity**, it strengthens connections when pre- and postsynaptic neurons increase or decrease together, independent of their absolute firing levels.

18.7 Oja's rule: Hebbian learning with normalization

As explained, pure Hebbian learning causes synaptic weights to grow without bound. To prevent this divergence, Oja proposed a stabilized Hebbian rule that automatically keeps the total weight magnitude under control [13]:

$$\tau_w \frac{d\vec{w}}{dt} = y \vec{x} - \alpha y^2 \vec{w} \quad (30)$$

where $\alpha > 0$ controls the strength of the normalization term. The postsynaptic activity is given by the linear readout $y = \sum_i w_i x_i$.

The rule contains two terms:

- **Hebbian term.** $y \vec{x}$: strengthens synapses that are correlated with the postsynaptic activity y .
- **Normalization term.** $\alpha y^2 \vec{w}$: scales weights down in proportion to their size, preventing runaway growth.

The key effect of the second term is that the total weight magnitude approaches a constant value.

Why Oja's rule normalizes the weights

We define the squared norm of the weight vector:

$$\|\vec{w}\|^2 = \sum_i w_i^2$$

To determine whether learning causes the weights to grow, shrink, or remain stable, we examine how this quantity evolves over time.

$$\frac{d}{dt} \|\vec{w}\|^2 = \frac{d}{dt} \sum_i w_i^2 = \sum_i 2w_i \frac{dw_i}{dt} = 2 \sum_i w_i \frac{dw_i}{dt}$$

Using Oja's rule:

$$\tau_w \frac{dw_i}{dt} = yx_i - \alpha y^2 w_i$$

we substitute into the expression above:

$$\tau_w \frac{d}{dt} \|\vec{w}\|^2 = 2 \sum_i w_i (yx_i - \alpha y^2 w_i)$$

Now distribute the sum:

$$\tau_w \frac{d}{dt} \|\vec{w}\|^2 = 2y \sum_i w_i x_i - 2\alpha y^2 \sum_i w_i^2$$

Recall that the postsynaptic activity is given by:

$$y = \sum_i w_i x_i$$

so we obtain:

$$\tau_w \frac{d}{dt} \|\vec{w}\|^2 = 2y^2 - 2\alpha y^2 \|\vec{w}\|^2 = 2y^2 (1 - \alpha \|\vec{w}\|^2)$$

Thus:

$$\begin{aligned} \frac{d}{dt} \|\vec{w}\|^2 &> 0 & \text{if } \|\vec{w}\|^2 < \frac{1}{\alpha} \\ \frac{d}{dt} \|\vec{w}\|^2 &< 0 & \text{if } \|\vec{w}\|^2 > \frac{1}{\alpha} \end{aligned}$$

Therefore the norm converges to:

$$\|\vec{w}\|^2 = \frac{1}{\alpha}$$

This means that Oja's rule forces the weight vector to stay on the surface of a sphere with radius $\sqrt{1/\alpha}$.

To understand what this does, consider the simple case of two synapses w_1 and w_2 . The constraint:

$$w_1^2 + w_2^2 = \frac{1}{\alpha}$$

restricts the weight vector to a circle. Hebbian learning tries to pull the vector toward inputs that correlate strongly with the output y , but the **normalization term keeps the vector on the circle**. The two effects together create **competition**: if input x_1 is more correlated with y than x_2 , then w_1 grows and w_2 shrinks until the vector points almost entirely in the w_1 direction.

This competition has an important consequence. Under standard assumptions (e.g. zero-mean inputs and slow learning), Oja's rule causes the weight vector to align with the direction in input space that exhibits the largest variance. In other words, the neuron learns the **first principal component** of its inputs, the dominant pattern of statistical structure.

18.8 The BCM rule: Hebbian learning with a sliding threshold

Oja's rule stabilizes Hebbian learning but still cannot make a **neuron selective for one particular input pattern**. To explain how neurons develop sharp receptive fields, we introduce the BCM rule.

If several inputs produce **similar postsynaptic activity, their synapses all potentiate together**. Nothing in Hebbian or covariance learning suppresses weaker patterns or favours the strongest one, so all correlated inputs grow at the same time. As a result, neither mechanism can produce input selectivity on its own.

The BCM rule [14] solves this by introducing a **sliding plasticity threshold** θ that moves according to the neuron's long-term activity. The learning rule is:

$$\boxed{\tau_w \frac{d\vec{w}}{dt} = y \vec{x} (y - \theta)} \quad (31)$$

and the threshold adapts slowly:

$$\tau_\theta \frac{d\theta}{dt} = y^2 - \theta \quad (32)$$

Note. Some formulations include a normalization term, $\tau_\theta \frac{d\theta}{dt} = y^2/y_{\text{target}} - \theta$, which simply rescales the adaptation rate without changing the qualitative behavior. Interpretation:

- If $y < \theta \Rightarrow$ synapses **depress**.
- If $y > \theta \Rightarrow$ synapses **potentiate**.
- θ increases when the neuron fires strongly, and decreases when activity is weak.

Thus, the neuron learns where to place the boundary between “too little” and “enough” activity. This sliding threshold is the key mechanism that produces selectivity.

Visual intuition. Figure 27 shows the BCM plasticity curve.

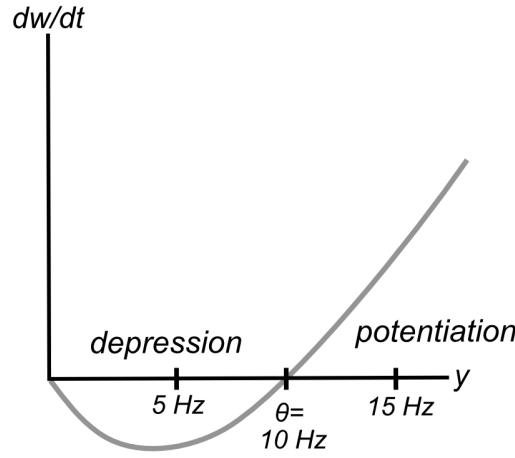


Figure 27: BCM plasticity curve. When the postsynaptic rate y is below the threshold θ , synapses depress; when $y > \theta$, they potentiate. The crossing point $y = \theta$ marks where the sign of learning reverses.

At low firing rates, $\frac{dw}{dt} < 0$ (depression). At high firing rates, $\frac{dw}{dt} > 0$ (potentiation). During learning, θ slides to track the neuron’s typical activity.

Why the threshold must slide. Consider two presynaptic inputs x_1 and x_2 :

$$y = w_1 x_1 + w_2 x_2$$

Suppose each input alone produces a different postsynaptic firing rate:

$$x_1 \text{ active} \Rightarrow y = 5 \text{ Hz}, \quad x_2 \text{ active} \Rightarrow y = 3 \text{ Hz}$$

If the threshold θ were fixed (e.g. $\theta = 10 \text{ Hz}$), then both firing rates always satisfy $y < \theta$, so both synapses would continually depress. The neuron would eventually become silent, and no selectivity could emerge. BCM avoids this failure because the threshold θ is not fixed, it slides.

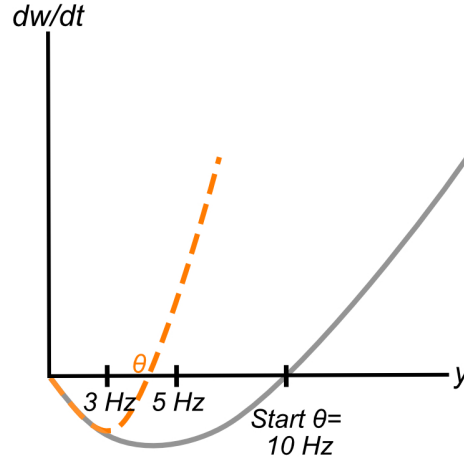


Figure 28: Sliding-threshold mechanism in BCM. The grey curve shows $\frac{dw}{dt}$ as a function of y . The orange dashed curve shows how θ decreases during learning. Starting from a high initial value (e.g. 10 Hz), the threshold moves toward the firing rates produced by the two patterns (3 Hz and 5 Hz), eventually settling *between* them. The stronger input (5 Hz) produces $y > \theta$ and potentiates; the weaker (3 Hz) produces $y < \theta$ and depresses. Only one pattern “wins.”

After learning (Figure 28):

- The 5 Hz input lies above the final threshold $\rightarrow$ its synapses potentiate.
- The 3 Hz input lies below threshold $\rightarrow$ its synapses depress.

The threshold stops exactly between 3 Hz and 5 Hz. Only the stronger input stays above threshold, making the neuron selective for that pattern.

Table 3: Summary of Hebbian-type learning rules and their functional consequences.

Learning rule	Fixes instability?	Creates selectivity?
Hebbian	No: Weights grow without bound	No: All co-active inputs strengthen
Covariance	No: Only baseline activity is removed	No: All correlated inputs still potentiate
Oja	Yes: Normalization keeps weights bounded	No: Competition is weak; multiple inputs persist
BCM	Yes: Sliding threshold stabilizes activity	Yes: Strong competition selects one input pattern

18.9 Spike-Timing-Dependent Plasticity

So far, we have studied **rate-based** learning rules (Hebbian, covariance, Oja, BCM). However, real neurons communicate using **spikes**, not continuous firing rates. Experiments show that the precise timing between presynaptic and postsynaptic spikes determines whether a synapse is strengthened or weakened. This timing-based plasticity is called **Spike-Timing-Dependent Plasticity (STDP)**.

Here, potentiation and depression refer to changes in synaptic strength: potentiation corresponds to an increase in the synaptic weight, whereas depression corresponds to a decrease.

If a presynaptic spike occurs shortly before a postsynaptic spike ($\Delta t = t_{\text{post}} - t_{\text{pre}} > 0$):

$$\Delta w > 0 \quad (\text{long-term potentiation, LTP})$$

If the postsynaptic spike occurs before the presynaptic spike ($\Delta t < 0$):

$$\Delta w < 0 \quad (\text{long-term depression, LTD})$$

The magnitude of the synaptic change decays exponentially with the time difference:

$$\Delta w(\Delta t) = \begin{cases} A_+ \exp(-\Delta t/\tau_+) & \Delta t > 0, \\ -A_- \exp(\Delta t/\tau_-) & \Delta t < 0. \end{cases}$$

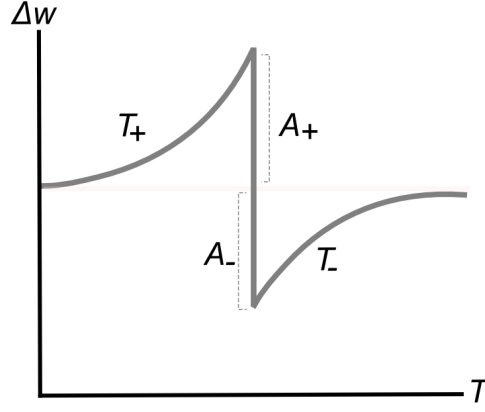


Figure 29: The classical STDP learning window. Presynaptic spikes arriving shortly before a postsynaptic spike produce potentiation (LTP), while post-before-pre timing produces depression (LTD). The magnitude decays exponentially with spike-time difference.

Eligibility-trace formulation. A convenient and widely used model of STDP introduces **eligibility traces**, which are variables that temporarily store information about recent spikes. They act as short-term memories that tell the synapse *how recently* a neuron has fired.

We denote:

- $z^{\text{pre}}(t)$: presynaptic eligibility trace.
- $z^{\text{post}}(t)$: postsynaptic eligibility trace.

These traces obey simple linear differential equations:

$$\tau_{\text{pre}} \frac{dz^{\text{pre}}}{dt} = -z^{\text{pre}} + \delta(t - t_{\text{pre}}) \quad \tau_{\text{post}} \frac{dz^{\text{post}}}{dt} = -z^{\text{post}} + \delta(t - t_{\text{post}})$$

Each spike produces an instantaneous jump in the corresponding trace, followed by exponential decay. This is the same forget-and-converge equation described at the initial chapters.

The synaptic weight is updated only at spike times:

$$\boxed{\frac{dw}{dt} = A_+ z^{\text{pre}} \delta(t - t_{\text{post}}) - A_- z^{\text{post}} \delta(t - t_{\text{pre}})} \quad (33)$$

Interpretation.

- At a **postsynaptic spike**, the presynaptic trace z^{pre} determines the amount of potentiation (LTP).
- At a **presynaptic spike**, the postsynaptic trace z^{post} determines the amount of depression (LTD).

Step-by-step intuition. Assume both neurons are initially silent.

- At time t_{pre} , the presynaptic neuron spikes. The trace z^{pre} jumps to 1 and then decays exponentially. No weight change occurs yet.
- At time $t_{\text{post}} > t_{\text{pre}}$, the postsynaptic neuron spikes. The presynaptic trace has value:

$$z^{\text{pre}}(t_{\text{post}}) = \exp\left(-\frac{t_{\text{post}} - t_{\text{pre}}}{\tau_{\text{pre}}}\right)$$

The weight increases by $A_+ \exp(-\Delta t / \tau_{\text{pre}})$ exactly reproducing the STDP learning window.

If the order of spikes is reversed, the same logic produces depression. Thus, the eligibility-trace formulation provides a direct mechanistic explanation of STDP.

What can neurons learn with STDP?

Preference for earlier inputs. If multiple presynaptic neurons fire in a consistent temporal order, STDP strengthens synapses of neurons that fire *just before* the postsynaptic spike. The neuron therefore becomes selective for inputs occurring earlier in time.

Learning temporal sequences. If neurons are repeatedly activated in a sequence:

$$N_1 \rightarrow N_2 \rightarrow N_3 \rightarrow N_4$$

STDP strengthens forward connections and weakens backward ones. This creates directed chains of connectivity, allowing networks to encode temporal sequences, motor patterns, and ordered memories.

STDP therefore complements rate-based Hebbian rules by providing a biologically realistic mechanism for learning **time-sensitive** structure.

19 Supervised Learning

In the previous chapter we focused on **unsupervised learning rules**, where synaptic changes depend only on neural activity itself. In many situations, however, learning is guided by an external signal that indicates whether the neuron's output is correct or incorrect. This learning paradigm is known as **supervised learning**.

In supervised learning, each input pattern $\vec{x}^n$ is paired with a **desired (target) output** y_n^t . The goal of learning is to adjust the synaptic weights so that the neuron's actual output y_n matches the target output y_n^t across all training examples.

19.1 The Perceptron

The simplest and most influential model of supervised learning is the **perceptron** [15]. It consists of a single neuron that learns to classify input patterns by adjusting its synaptic weights using an explicit teaching signal.

For each input pattern $\vec{x}^n = (x_1^n, \dots, x_N^n)$, the perceptron computes a weighted sum of its inputs and compares it to a threshold:

$$y_n = \Theta \left(\sum_{i=1}^N w_i x_i^n - b \right)$$

where $\vec{w} = (w_1, \dots, w_N)$ are the synaptic weights, b is a bias (or threshold), and $\Theta(\cdot)$ is the Heaviside step function, defined as:

$$\Theta(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

The perceptron therefore produces a **binary output**: it either fires ($y_n = 1$) or remains silent ($y_n = 0$).

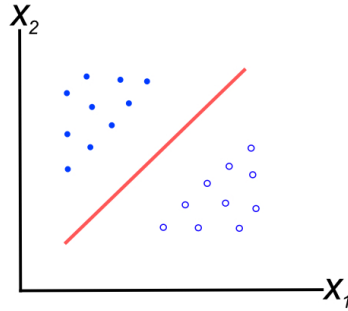


Figure 30: Structure of a perceptron. Input features are linearly combined through synaptic weights and compared to a threshold.

Learning proceeds by comparing the actual output y_n to the target output y_n^t . A simple measure of the error for pattern n is

$$E_n = (y_n^t - y_n)^2$$

To reduce this error, synaptic weights are updated according to the **perceptron learning rule**. In discrete time, the update takes the form:

$$\Delta w_i = \eta (y_n^t - y_n) x_i^n$$

where $\eta > 0$ is the learning rate. In continuous time, this can be written as:

$$\tau_w \frac{dw_i}{dt} = x_i^n (y_n^t - y_n) \quad (34)$$

This rule has a simple interpretation, which follows directly from the error term $(y_n^t - y_n)$. If the neuron fires when it should not ($y_n = 1, y_n^t = 0$), the error is negative and synapses associated with active inputs are weakened. If the neuron fails to fire when it should ($y_n = 0, y_n^t = 1$), the error is positive and those synapses are strengthened. When the output is already correct ($y_n = y_n^t$), the error vanishes and no learning occurs.

19.2 Geometric interpretation

Geometrically, the perceptron implements a **linear classifier**. The condition:

$$\sum_{i=1}^N w_i x_i - b = 0$$

defines a hyperplane in input space that separates patterns producing $y_n = 1$ from those producing $y_n = 0$. As learning proceeds, changes in the synaptic weights rotate and shift this hyperplane so as to correctly separate the training examples.

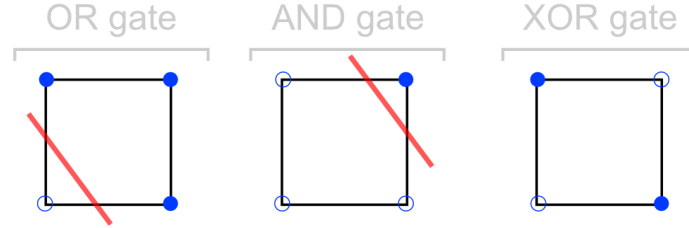


Figure 31: Geometric interpretation of logic gates using a perceptron. AND and OR problems are linearly separable and can be solved by a single perceptron, whereas XOR is not.

As a consequence, a single perceptron can learn linearly separable problems such as AND and OR, but it cannot solve the XOR problem. Intuitively, XOR requires combining multiple decision boundaries: the inputs must first be separated into intermediate features, which are then recombined to produce the correct output. This can be achieved by stacking perceptrons in multiple layers, motivating multilayer neural networks.

19.3 Relation to biological learning

Although highly simplified, the perceptron learning rule captures several ideas that are relevant for biological synaptic plasticity. **Weight updates depend on presynaptic activity, as in Hebbian learning, while the target signal plays the role of a teaching or error signal.** Closely related supervised learning mechanisms have been proposed in models of cerebellar learning. The perceptron therefore provides a conceptual bridge between biological learning rules and modern supervised learning algorithms.

20 Reinforcement Learning

In supervised learning, each input pattern is paired with an explicit target output. In many real-world situations, however, such targets are not available. Instead, an agent receives only a **reward signal** indicating whether an outcome was good or bad. Learning from reward rather than explicit targets is known as **reinforcement learning**.

Rather than asking “what is the correct output?”, reinforcement learning asks “How good was the outcome?” and uses this information to guide future behavior.

20.1 Classical conditioning

A canonical example of reinforcement learning comes from **classical conditioning** experiments. Initially, an animal shows an innate response to food, but not to a neutral sound:

Food $\rightarrow$ Salivation

Sound $\rightarrow$ No salivation

During training, the neutral stimulus is repeatedly paired with the reward:

Sound + Food $\rightarrow$ Salivation

After sufficient training, the sound alone becomes predictive of the reward:

Sound $\rightarrow$ Salivation

The animal has learned to **predict reward** (food) from a previously neutral stimulus.

If the sound is later presented repeatedly without food:

Sound $\rightarrow$ No food

the conditioned response gradually disappears:

Sound $\rightarrow$ No salivation

This process is known as **extinction**. Together, conditioning and extinction show that learning depends not simply on stimulus–reward pairing, but on how well reward is *predicted*.

20.2 The Rescorla–Wagner model

A simple and influential model capturing these ideas is the **Rescorla–Wagner rule**. We consider a binary stimulus $u \in \{0, 1\}$ (e.g. sound absent or present), a reward r (e.g. food), and a learned prediction of reward, given by:

$$v = w u$$

The objective of learning is to make the predicted reward v match the actual reward r . A simple measure of the resulting prediction error is:

$$E = (r - v)^2$$

Learning proceeds by minimizing this error. Using gradient descent, the synaptic weight is adjusted in the direction that most rapidly reduces E . The gradient $\partial E / \partial w$ quantifies how sensitively the error changes when the weight is modified, leading to the update rule:

$$\Delta w = \eta(r - v) u$$

where $\eta > 0$ is the learning rate.

The quantity:

$$\delta = r - v$$

is called the **prediction error**.

This rule admits a simple and intuitive interpretation. If the reward is larger than expected ($\delta > 0$), the association between stimulus and reward is strengthened. If the reward is smaller than expected ($\delta < 0$), the association is weakened. When the reward is perfectly predicted ($\delta = 0$), learning ceases. Despite its simplicity, this rule successfully reproduces both conditioning and extinction observed experimentally.

20.3 Blocking and limitations

The Rescorla–Wagner model also explains the phenomenon of **blocking**. Suppose a stimulus S_1 has already been learned and perfectly predicts reward. In this case, the predicted reward matches the actual reward, so the prediction error $\delta = r - v$ is zero.

If a second stimulus S_2 is now presented together with S_1 , the reward is already fully predicted by S_1 . Because the prediction error remains zero, there is no learning signal available to update the association for S_2 . As a result, S_2 fails to acquire predictive value and is said to be **blocked**.

However, not all learning phenomena can be explained within this framework. In particular, effects such as **secondary conditioning** highlight an **important limitation of the Rescorla–Wagner model: time is not explicitly represented**. Because learning depends only on immediate prediction error, the model cannot account for situations in which predictive relationships unfold across time.

20.4 Temporal-Difference learning

The key limitation of the **Rescorla–Wagner model is that it treats learning as instantaneous**: predictions and rewards are compared at the same moment in time. In many real tasks, however, **rewards arrive only after a delay**, and learning must assign credit to stimuli that occurred earlier.

Temporal-Difference (TD) learning addresses this limitation by introducing a **value function**, which represents the expected total future reward from a given time:

$$V(t) = \text{expected total future reward from time } t$$

In practice, the value function is often modeled as a weighted sum of past stimuli, allowing the system to distribute credit across time by remembering which inputs preceded reward:

$$V(t) = \sum_{\tau=0}^{\infty} w(\tau) u(t - \tau) \quad (35)$$

Formally, the value function approximates the sum of future rewards:

$$V(t) \approx \sum_{\tau=0}^{\infty} r(t + \tau)$$

Rather than comparing predictions directly to rewards, TD learning compares successive predictions. This leads to the temporal-difference error:

$$\delta(t) = r(t) + V(t + 1) - V(t) \quad (36)$$

This error is nonzero whenever the prediction of future reward changes from one time step to the next.

Learning updates the weights according to:

$$w \leftarrow w + \eta \delta(t) u(t)$$

The crucial idea is that **learning occurs whenever predictions about future reward change**. By propagating prediction errors backward in time, TD learning can account for delayed rewards, chains of predictors, and secondary conditioning.

Remarkably, temporal-difference learning aligns closely with biological data. **Dopamine neurons** in the brain appear to encode a reward prediction error. Before learning, dopamine activity responds to unexpected reward. After learning, it shifts to the predictive stimulus. If an expected reward is omitted, dopamine activity is suppressed.

This striking correspondence suggests that reinforcement learning principles may be directly implemented by neural circuits.

21 Hopfield networks and associative memory

A **Hopfield network** is a simple model of a neural network that can **store and retrieve memories**. The key idea is that **memories are stored in the connections between neurons**, and retrieved through the natural dynamics of the network.

The model consists of many simple neurons interacting with each other, and its behavior is inspired by ideas from statistical physics and magnetism.

21.1 Binary neurons and interactions

We consider a network of N neurons. Each neuron can be in one of two states:

$$s_i \in \{-1, +1\}$$

You can think of $s_i = +1$ as an active neuron and $s_i = -1$ as an inactive one.

Neurons influence each other through synaptic weights w_{ij} . The weight w_{ij} measures how much neuron j affects neuron i . All neurons are connected to each other.

The total input (also called the local field) received by neuron i at time t is:

$$h_i(t) = \sum_{j=1}^N w_{ij} s_j(t)$$

This quantity simply sums the contributions from all other neurons in the network.

21.2 Neuron update rule

Each neuron updates its state based on the total input it receives from the rest of the network. Recall that the state of neuron i is a binary variable:

$$s_i \in \{-1, +1\}$$

and that $h_i(t)$ denotes the total input (local field) received by neuron i at time t . In the deterministic case, the update rule is:

$$s_i(t + \Delta t) = \text{sign}(h_i(t))$$

This equation states that the **future state of neuron i depends only on the sign of its total input**. More explicitly:

- If $h_i(t) > 0$, then $s_i(t + \Delta t) = +1$ and the neuron becomes active
- If $h_i(t) < 0$, then $s_i(t + \Delta t) = -1$ and the neuron becomes inactive

Each neuron therefore updates its state by aligning with the dominant influence of the other neurons in the network. The collective evolution of all neuron states defines the dynamics of the Hopfield network.

So far, we have assumed perfectly deterministic neuron updates.

Stochastic update rule (with noise) In many situations, neuron updates are not perfectly deterministic. Instead, **neurons update their state probabilistically**, with a probability that depends on the total input they receive.

In this case, the probability that neuron i becomes active at the next time step is:

$$\Pr(s_i(t + \Delta t) = +1 \mid h_i(t)) = g(h_i(t))$$

The probability that the neuron becomes inactive is given by $1 - g(h_i(t))$. The function $g(\cdot)$ is typically chosen as a sigmoidal function:

$$g(h) = \frac{1}{2} [1 + \tanh(\beta h)]$$

The parameter β controls the level of noise in the system. Large values of β correspond to low noise and nearly deterministic updates, while small values of β correspond to high noise and more random behavior. In the limit $\beta \rightarrow \infty$, the stochastic update rule reduces to the deterministic sign rule.

21.3 Detour: connection to magnetism

The Hopfield network was originally inspired by models of magnetism in physics. In this analogy, **each neuron state $s_i \in \{-1, +1\}$ is interpreted as a spin that can point in one of two directions.**

Interactions between neurons correspond to interactions between spins, and the collective behavior of the network resembles the behavior of magnetic materials.

21.3.1 Ferromagnetism

In a ferromagnetic material, neighboring spins tend to align with each other. **If most spins point in the same direction, the material becomes magnetized.**

If a **single spin is flipped** due to noise or an impurity, the **surrounding spins exert a strong influence that pulls it back into alignment.** As a result, the system returns to a uniform, ordered state.

In Hopfield networks, this corresponds to a situation in which **neurons prefer to have the same state.** Such interactions stabilize coherent patterns and make stored memories robust to small perturbations.

The strength of alignment depends on temperature. Low temperature corresponds to strong alignment and stable order, while high temperature introduces randomness and destroys magnetization.

21.3.2 Antiferromagnetism

In an antiferromagnetic material, **spins prefer to point in opposite directions to their neighbors.** This leads to **alternating patterns, such as checkerboard-like arrangements.**

Here, the system does not converge to a uniform state. Instead, **structure emerges from opposing interactions** between different groups of spins.

In the context of Hopfield networks, this illustrates how **specific interaction patterns can enforce structured configurations rather than global alignment.**

Back to neuroscience The key lesson from magnetism is that **global behavior emerges from local interactions.** By choosing synaptic weights carefully, Hopfield networks use the same principle to store memories as stable attractor states in neural activity.

21.4 Pattern storage in Hopfield networks

The main purpose of a Hopfield network is to **store and retrieve patterns** in an associative way. Each stored pattern represents a memory of the network.

A **pattern** is simply a particular configuration of the states of all N neurons in the network. We denote the stored patterns by:

$$\vec{p}^\mu = (p_1^\mu, p_2^\mu, \dots, p_N^\mu)$$

where:

- $\mu = 1, \dots, M$ labels the pattern.
- M is the total number of patterns we want to store.
- N is the number of neurons in the network.

Figure 32 illustrates this idea. Each row corresponds to one stored pattern, while each entry represents the state of a single neuron.

In the figure, neuron states are written using a **binary encoding** $p_i^\mu \in \{0, 1\}$, where 1 denotes an active neuron and 0 an inactive one.

For the theoretical analysis, it is often more convenient to use the equivalent **spin representation**:

$$p_i^\mu \in \{-1, +1\}$$

Both encodings describe the same patterns and can be converted into each other by a simple rescaling.

The task of the Hopfield network is to learn these M patterns and to store them as **stable states** of the network dynamics, so that each pattern can be recovered even when the network is initialized from a partial or noisy version of it.

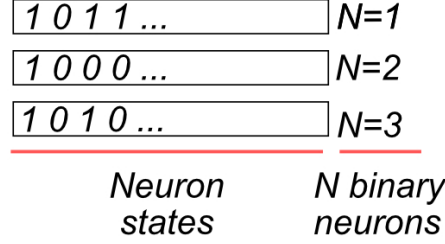


Figure 32: Representation of stored patterns in a Hopfield network. Each row corresponds to one pattern $\vec{p}^\mu$, while each entry represents the state of a single neuron. In this illustration, neuron states are written in binary form (0 = inactive, 1 = active). The total number of entries in each row equals the number of neurons N .

To store a pattern successfully, we want the following property to hold: **if the network is initialized exactly in a stored pattern, it should remain in that pattern forever.**

Mathematically, this means that each pattern $\vec{p}^\mu$ should be a **fixed point** of the dynamics:

$$s_i(t) = s_i(t + \Delta t) = p_i^\mu$$

In other words, once the network reaches a stored pattern, neuron states no longer change. To ensure that all stored patterns become **fixed points** of the network dynamics, the synaptic weights are chosen using the **Hopfield (Hebbian) learning rule**:

$$w_{ij} = c \sum_{\mu=1}^M p_i^\mu p_j^\mu \tag{37}$$

with the additional constraint (a neuron does not have a synapse onto itself):

$$w_{ii} = 0$$

Here, $c > 0$ is a constant that sets the overall strength of the synaptic connections.

This rule has a simple and intuitive interpretation:

- If two neurons are often in the **same state** across the stored patterns, their mutual connection becomes strong and positive.
- If two neurons are often in **opposite states**, their connection becomes negative.

As a result, each stored pattern reinforces itself under the network dynamics and remains stable once it is reached.

This learning rule is a direct mathematical expression of the principle:

“neurons that fire together wire together”

21.5 Pattern retrieval

After learning, the Hopfield network can retrieve stored patterns through its recurrent dynamics. We initialize the network in some state:

$$\vec{s}(t_0) = \{s_i(t_0)\}$$

which may be a noisy or incomplete version of a stored pattern.

As the neurons update their states, the dynamics should converge to a **fixed point** corresponding to the stored pattern that is most similar to the initial state. To quantify similarity, we define the **overlap with pattern μ** :

$$m^\mu(t) = \frac{1}{N} \sum_{i=1}^N p_i^\mu s_i(t) \quad (38)$$

The overlap measures how well the current network state matches pattern μ : larger $m^\mu(t)$ means greater similarity.

Using the Hopfield learning rule:

$$w_{ij} = c \sum_{\mu=1}^M p_i^\mu p_j^\mu$$

the total input (local field) to neuron i is:

$$h_i(t) = \sum_j w_{ij} s_j(t) = cN \sum_{\mu=1}^M p_i^\mu m^\mu(t)$$

Choosing $c = 1/N$ gives:

$$h_i(t) = \sum_{\mu=1}^M p_i^\mu m^\mu(t)$$

The neuron update rule then becomes:

$$s_i(t + \Delta t) = \text{sign}(h_i(t))$$

If one overlap $m^\mu(t)$ is larger than the others, its contribution dominates the sum, and the network state is driven toward the corresponding stored pattern $\vec{p}^\mu$.

Thus, pattern retrieval is completely determined by the overlaps $m^\mu(t)$.

Example of memory retrieval Consider a Hopfield network storing three patterns:

$$\mu = 1, 2, 3$$

Assume the initial state has a nonzero overlap only with pattern $\mu = 3$:

$$m^3(t_0) = 0.4, \quad m^1(t_0) = m^2(t_0) = 0$$

In the deterministic case, the neuron update rule is:

$$s_i(t_0 + \Delta t) = \text{sign} \left(\sum_{\mu=1}^3 p_i^\mu m^\mu(t_0) \right)$$

Since only $m^3(t_0)$ is nonzero, this reduces to:

$$s_i(t_0 + \Delta t) = \text{sign}(p_i^3 m^3(t_0))$$

Because $m^3(t_0) > 0$, the sign is determined by p_i^3 , and thus:

$$s_i(t_0 + \Delta t) = p_i^3 \quad \text{for all } i$$

After a single time step, the network reaches the stored pattern $\vec{p}^3$, because the update already reaches a fixed point and further updates do not change the state.

References

- [1] Alan L Hodgkin and Andrew F Huxley. The components of membrane conductance in the giant axon of Loligo. *The Journal of Physiology*, 116:473–496, 1952.
- [2] Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 1952.
- [3] Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961.
- [4] Jinichi Nagumo, Suguru Arimoto, and Shosaku Yoshizawa. An active pulse transmission line simulating nerve axon. *Proceedings of the IRE*, 50(10):2061–2070, 1962.
- [5] Catherine Morris and Harold Lecar. Voltage oscillations in the barnacle giant muscle fiber. *Biophysical Journal*, 35(1):193–213, 1981.
- [6] Eugene M Izhikevich. Simple model of spiking neurons. *IEEE Transactions on Neural Networks*, 14(6):1569–1572, 2003.
- [7] Romain Brette and Wulfram Gerstner. Adaptive exponential integrate-and-fire model as an effective description of neuronal activity. *Journal of Neurophysiology*, 94(5):3637–3642, 2005.
- [8] Wulfram Gerstner, Werner M Kistler, Richard Naud, and Liam Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.
- [9] Michael Okun and Ilan Lampl. Instantaneous correlation of excitation and inhibition during ongoing and sensory-evoked activities. *Nature Neuroscience*, 11(5):535–537, 2008.
- [10] Hirofumi Ozeki, Ian M Finn, Evan S Schaffer, Kenneth D Miller, and David Ferster. Inhibitory stabilization of the cortical network underlies visual surround suppression. *Neuron*, 62(4):578–592, 2009.
- [11] Raffi Ben-Yishai, Rotem L Bar-Or, and Haim Sompolinsky. Theory of orientation tuning in visual cortex. *Proceedings of the National Academy of Sciences*, 92(9):3844–3848, 1995.
- [12] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, 1949.
- [13] Erkki Oja. Simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, 15(3):267–273, 1982.
- [14] Elie L Bienenstock, Leon N Cooper, and Paul W Munro. Theory for the development of neuron selectivity: orientation specificity and binocular interaction in visual cortex. *Journal of Neuroscience*, 2(1):32–48, 1982.
- [15] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.